Package 'DrugSigNet'

September 4, 2026

Type Package

Title Drug Signature Network

Version 1.0.0

Maintainer Orhan Bellur <orhan.bellur@helmholtz-muenchen.de>

Description R package to conduct in silico drug repurposing analysis by leveraging several algorithms for both signature-based and network-based approaches.

License MIT + file LICENSE

URL https://github.com/compneurobio/DrugSigNet

BugReports https://github.com/compneurobio/DrugSigNet/issues

Imports BoutrosLab.plotting.general,
rcdk,
dplyr,
ggsci,
ggplot2,
ggraph,
igraph,
magrittr,
methods,
openxlsx,
parallel,
purrr,
reticulate,
rlang,
RobustRankAggreg,
scales,
signatureSearch,
stats,
stringr,
tibble,
tidyr,
tidyselect,
utils

Suggests AnnotationDbi,
ExperimentHub,
HDF5Array,
enrichR,
ggalluvial,
httr,
kableExtra,
knitr,
plotly,
pkgload,
psych,
readr,
org.Hs.eg.db,
rhdf5,
remotes,
rmarkdown,
testthat,
tinytex,
webr,
wordcloud

Enhances synapser

VignetteBuilder knitr

Config/testthat/edition 3

Encoding UTF-8

LazyData true

Depends R (>= 3.5)

Config/roxygen2/version 8.0.0

Contents

`annotate_drugs` *Retrieve Drug Annotations*

Description

Retrieves and combines drug annotations from one or more supported data sources, including drug synonyms, mechanisms of action, SMILES, blood-brain barrier permeability, ATC classification, indications, approval status, adverse events, and optionally clinical trial information.

Usage

```
annotate_drugs(
  object = NULL,
  drugs = NULL,
  source = c("All", "CHEMBL", "OpenTargets", "TTD"),
  condition = NULL,
  force = FALSE,
  auth_token = NULL
)
```

Arguments

`object`	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`drugs`	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`source`	Annotation source. One of ‘"All"‘, ‘"CHEMBL"‘, ‘"OpenTargets"‘, or ‘"TTD"‘.

condition	Optional disease or indication used to retrieve matching clinical trial information.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

Annotation layers unavailable for the selected source are skipped automatically. Available layers include drug synonyms, approval status, indications, mechanism of action, blood-brain barrier permeability, ATC classification, adverse events, SMILES structures, and clinical trials when 'condition' is provided.

Value

A data frame containing one row per drug with all available annotation fields merged across the selected data source(s).

Examples

```
## Not run:
annotate_drugs(
  drugs = c("metformin", "donepezil", "nifedipine"),
  source = "All"
)

drug_df <- data.frame(
  Drug = c("metformin", "donepezil", "nifedipine")
)

annotate_drugs(
  object = drug_df,
  source = "CHEMBL"
)

annotate_drugs(
  drugs = c("metformin", "donepezil"),
  source = "All",
  condition = "Alzheimer disease"
)

## End(Not run)
```

`calculate_rank_aggregation` *Aggregate signature rank tables with one rank aggregation method*

Description

This implementation is intentionally compatible with the original Harmonize_Signature_Network() signature harmonization behaviour: - input rows are preserved as-is (no duplicate collapsing), - missing rank_score values remain NA (no NA -> 0 replacement), - the identifier remains 'perturbation' during the first aggregation stage, - rank input columns are generated dynamically as rank_score_1 ... rank_score_n.

Usage

```
calculate_rank_aggregation(
  method,
  ...,
  rank_inputs = NULL,
  prior = 0.093,
  num_bin = 200,
  ties_method = "max"
)
```

Arguments

`method`	Rank aggregation method. One of `"CRank"`, `"Dowdall"`, or `"RRA"`.
`...`	Processed signature rank tables containing `perturbation` and `rank_score` columns.
`rank_inputs`	Optional list of processed signature rank tables. When supplied, `...` is ignored.
`prior`	Prior passed to `CRank()`.
`num_bin`	Number of bins passed to `CRank()`.
`ties_method`	Ties method passed to rank aggregation methods.

Value

A `RankAggregation` object.

cell_info *Cell-Line Metadata*

Description

Metadata for cell lines represented in the bundled connectivity-map reference data.

Format

A data frame with one row per cell line and columns describing the cell-line identifier and biological attributes.

cell_info2 *Extended Cell-Line Metadata*

Description

An extended cell-line metadata table used with LINCS signatures.

Format

A data frame with one row per cell line and columns describing the cell-line identifier and biological attributes.

Note

This object is distributed in its original binary data file.

check_drugsignet_installation *Check DrugSigNet Installation Health*

Description

Performs a lightweight post-installation smoke check for DrugSigNet. This is useful after installing from Git with ‘remotes::install_git()‘ to confirm that core R dependencies, key exported functions, optional Synapse support, and Python modules used by reticulate-backed workflows are available.

Usage

```
check_drugsignet_installation(
  check_python = TRUE,
  check_synapser = FALSE,
  check_optional = TRUE,
  install_missing_synapser = FALSE,
  stop_on_error = FALSE
)
```

Arguments

check_python	Logical; if 'TRUE', check Python modules used by DrugSigNet through 'reticulate'.
check_synapser	Logical; if 'TRUE', require the optional 'synapser' package to be installed and loadable.
check_optional	Logical; if 'TRUE', check optional reporting and plotting packages such as 'plotly', 'kableExtra', and 'tinytex'.
install_missing_synapser	Logical; if 'TRUE' and 'check_synapser = TRUE', attempt to install/load 'synapser' through the DrugSigNet Synapse helper before reporting check results.
stop_on_error	Logical; if 'TRUE', stop when any requested check fails.

Value

A list with logical 'ok' plus data frames for package, function, and Python module checks.

Examples

```
## Not run:
setup_synapser()
check_drugsignet_installation(check_synapser = TRUE, stop_on_error = TRUE)

## End(Not run)
```

---

chembl_moa_list *ChEMBL Mechanism-of-Action Annotations*

Description

Drug and mechanism-of-action annotations derived from ChEMBL.

Format

A data frame containing drug identifiers and their mechanism or target annotations.

---

clue_moa_list *CLUE Mechanism-of-Action Annotations*

Description

Drug and mechanism-of-action annotations used by the CLUE connectivity-map resources.

Format

A data frame containing perturbagen names and mechanism-of-action annotations.

cmap_method *CMAP Method for Signature-Based Drug Searching*

Description

Performs gene expression signature-based drug repurposing using the CMAP algorithm.

Usage

```
cmap_method(
  object = NULL,
  upset,
  downset,
  ref_db,
  chunk_size = 5000,
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  auth_token = NULL,
  validate_signature_refdb = TRUE
)
## S4 method for signature 'SignatureBased'
cmap_method(object)
```

Arguments

object	Optional ‘SignatureBased‘ object. If ‘NULL‘, a new object is created from ‘upset‘, ‘downset‘, and ‘ref_db‘.
upset	Character vector of up-regulated Entrez gene IDs. May be ‘NULL‘ if ‘downset‘ is provided.
downset	Character vector of down-regulated Entrez gene IDs. May be ‘NULL‘ if ‘upset‘ is provided.
ref_db	Reference database. Use ‘"cmap"‘ or ‘"lincs2"‘, or provide a local HDF5 path returned by ‘load_signature_refdb()‘ to use a frozen Synapse reference database.
chunk_size	Integer; number of reference signatures processed per chunk. Default is ‘5000‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.
auth_token	Optional Synapse authentication token used when ‘signature_refdb_mode‘ requests a frozen database. If ‘NULL‘, ‘SYNAPSE_AUTH_TOKEN‘ is used.
validate_signature_refdb	Logical; whether to validate frozen HDF5 reference databases before use. Default is ‘TRUE‘.

Details

The query may contain up-regulated genes (‘upset‘), down-regulated genes (‘downset‘), or both. At least one of ‘upset‘ or ‘downset‘ must be provided; they cannot both be ‘NULL‘.

‘ref_db‘ can be ‘"cmap"‘ or ‘"lincs2"‘. A local HDF5 path returned by ‘load_signature_refdb()‘ can also be supplied to use a frozen Synapse reference database. Frozen databases can also be fetched directly by setting ‘signature_refdb_mode = "frozen"‘ or ‘"frozen_force"‘ and providing ‘auth_token‘.

Value

A ‘SignatureBased‘ object containing the CMAP search results and query metadata.

Examples

```
## Not run:
upset <- c("7157", "1956", "5290")
downset <- c("7422", "4318", "348")

# Search using the default CMAP reference database
res <- cmap_method(
  upset = upset,
  downset = downset,
  ref_db = "cmap"
)

# Run a one-sided query
res_up <- cmap_method(
  upset = upset,
  downset = NULL,
  ref_db = "cmap"
)

# Fetch and use the frozen Synapse reference database directly
res_frozen <- cmap_method(
  upset = upset,
  downset = downset,
  ref_db = "cmap",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Reuse a local frozen HDF5 reference database
cmap_ref <- load_signature_refdb(
  ref_db = "cmap",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_local <- cmap_method(
  upset = upset,
  downset = downset,
```

```
  ref_db = cmap_ref
)

## End(Not run)
```

`cmap_rank_score` *Rank CMAP Signature Results*

Description

Converts raw results from [cmap_method()] into the standardized ranking format used by the signature pipeline.

Usage

```
cmap_rank_score(cmap_signature, confidence_level = 0.95, ties_method = "max")
```

Arguments

`cmap_signature`	A data frame of raw CMAP signature results.
`confidence_level`	Numeric confidence level for the Kolmogorov-Smirnov critical-value cutoff.
`ties_method`	Character string passed to [base::rank()], or ‘"dense"‘.

Value

A data frame of standardized CMAP rank scores.

`correlation_method` *Correlation Method for Signature-Based Drug Searching*

Description

Performs gene expression signature-based drug repurposing using a correlation-based search.

Usage

```
correlation_method(
  object = NULL,
  signature_matrix,
  ref_db,
  method = "spearman",
  chunk_size = 5000,
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  auth_token = NULL,
```

```
  validate_signature_refdb = TRUE
)

## S4 method for signature 'SignatureBased'
correlation_method(object)
```

Arguments

object	Optional ‘SignatureBased‘ object. If ‘NULL‘, a new object is created from ‘signature_matrix‘, ‘ref_db‘, and ‘method‘.
signature_matrix	Numeric matrix containing log2 fold changes. The matrix must have one column, with gene identifiers stored as row names.
ref_db	Reference database. Use ‘"cmap"‘ or ‘"lincs2"‘, or provide a local HDF5 path returned by ‘load_signature_refdb()‘ to use a frozen Synapse reference database.
method	Correlation method. One of ‘"spearman"‘, ‘"pearson"‘, or ‘"kendall"‘. Default is ‘"spearman"‘.
chunk_size	Integer; number of reference signatures processed per chunk. Default is ‘5000‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.
auth_token	Optional Synapse authentication token used when ‘signature_refdb_mode‘ requests a frozen database. If ‘NULL‘, ‘SYNAPSE_AUTH_TOKEN‘ is used.
validate_signature_refdb	Logical; whether to validate frozen HDF5 reference databases before use. Default is ‘TRUE‘.

Details

The method compares the input gene expression signature with signatures in a reference database using the selected correlation method. The input ‘signature_matrix‘ must be a numeric matrix containing log2 fold changes, with one column and gene identifiers stored as row names.

‘ref_db‘ can be ‘"cmap"‘ or ‘"lincs2"‘. A local HDF5 path returned by ‘load_signature_refdb()‘ can also be supplied to use a frozen Synapse reference database. Frozen databases can also be fetched directly by setting ‘signature_refdb_mode = "frozen"‘ or ‘"frozen_force"‘ and providing ‘auth_token‘.

Value

A ‘SignatureBased‘ object containing the correlation search results and query metadata.

Functions

- `correlation_method(SignatureBased)`: Implements the Correlation method for Signature-Based Drug Searching.

Examples

```
## Not run:
# Differential gene expression signature (log2 fold changes)
signature_matrix <- matrix(
  c(1.35, -0.82, 2.11, -1.47),
  ncol = 1,
  dimnames = list(
    c("7157", "1956", "5290", "7422"),
    "log2FC"
  )
)

# Search using the default CMAP reference database
res <- correlation_method(
  signature_matrix = signature_matrix,
  ref_db = "cmap"
)

# Use Pearson correlation instead of the default Spearman correlation
res_pearson <- correlation_method(
  signature_matrix = signature_matrix,
  ref_db = "cmap",
  method = "pearson"
)

# Automatically fetch and use a frozen Synapse reference database
res_frozen <- correlation_method(
  signature_matrix = signature_matrix,
  ref_db = "cmap",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Reuse a local frozen HDF5 reference database
cmap_ref <- load_signature_refdb(
  ref_db = "cmap",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_local <- correlation_method(
  signature_matrix = signature_matrix,
  ref_db = cmap_ref
)

## End(Not run)
```

`correlation_rank_score`

Rank Correlation Signature Results

Description

Converts raw results from [correlation_method()] into the standardized ranking format used by the signature pipeline.

Usage

```
correlation_rank_score(correlation_signature, n, ties_method = "max")
```

Arguments

correlation_signature	A data frame of raw correlation results.
n	Number of observations used to calculate correlation significance.
ties_method	Character string passed to [base::rank()], or ‘"dense"‘.

Value

A data frame of standardized correlation rank scores.

CRank *CRank Rank Aggregation Method*

Description

Aggregates multiple ranked drug lists using the CRank algorithm.

Usage

```
CRank(
  object = NULL,
  input_data,
  ties_method = "max",
  prior = 0.093,
  num_bin = 200,
  num_iter = 1000,
  reverse = FALSE
)
## S4 method for signature 'RankAggregation'
CRank(object)
```

Arguments

object	Optional ‘RankAggregation‘ object. For pipe-friendly use, this can also be a data frame used as ‘input_data‘.
input_data	Data frame containing items and ranking scores. The first column should contain item identifiers, and at least two additional columns must be numeric.

ties_method	Method used to resolve ties during rank conversion. One of ‘"max"‘, ‘"min"‘, ‘"average"‘, ‘"first"‘, ‘"last"‘, ‘"random"‘, or ‘"dense"‘. Default is ‘"max"‘.
prior	Numeric prior used by CRank. Default is ‘0.093‘.
num_bin	Number of bins used by CRank. Default is ‘200‘.
num_iter	Number of CRank iterations. Default is ‘1000‘.
reverse	Logical; if ‘TRUE‘, smaller input values are treated as better, which is appropriate when numeric columns already represent ranks. If ‘FALSE‘, larger input values are treated as better. Default is ‘FALSE‘.

Details

‘CRank()‘ combines rankings from at least two numeric columns in ‘input_data‘. The first column is treated as the item identifier, typically ‘Drug‘, and the remaining numeric columns are converted to ranks before running the Python-based CRank implementation.

CRank is a Bayesian rank aggregation algorithm that estimates a consensus ranking from multiple input rankings. It models the probability that an item belongs to the top-ranked group across different methods and uses iterative inference to update this probability until convergence. Items consistently ranked highly across methods receive better consensus ranks.

In DrugSigNet, CRank is used to integrate evidence across multiple drug-ranking methods, such as signature-based or network-based scores, into a single consensus ranking.

Missing numeric values are replaced with zero before ranking. Values equal to zero are treated as unranked entries and remain zero in the CRank input.

For pipe-friendly use, a data frame can be supplied as ‘object‘; in that case, it is used as ‘input_data‘.

Value

A ‘RankAggregation‘ object containing the aggregated CRank results in ‘object@result‘ and convergence information in ‘object@parameters‘.

References

Zitnik M, Sosič R, Leskovec J. Prioritizing network communities. *Nature Communications*. 2018;9:2544. doi:10.1038/s41467018049485

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3)
)

res <- CRank(
  input_data = rank_df,
  ties_method = "max",
  prior = 0.093,
  num_bin = 200,
```

```
  num_iter = 1000,
  reverse = TRUE
)

# Pipe-friendly use
res_pipe <- CRank(rank_df, reverse = TRUE)

## End(Not run)
```

`createGraphNetwork` *Create Complete Graph Network*

Description

Integrates PPI and drug-target graphs into a complete network.

Usage

```
createGraphNetwork(
  drug_node_edge,
  gene_node_edge,
  drug_node,
  gene_node,
  file_name
)
```

Arguments

`drug_node_edge`	Drug node-to-edge mappings.
`gene_node_edge`	Gene node-to-edge mappings.
`drug_node`	Drug node data.
`gene_node`	Gene node data.
`file_name`	Output file name for the graph network.

`createNodeToEdge` *Create Node-to-Edge Graphs*

Description

Creates node-to-edge mappings for PPI and drug-target networks.

Usage

```
createNodeToEdge(ppi_network, drug_target_network)
```

Arguments

ppi_network	Protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default ‘"gene_gene"‘ network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default ‘"drug_target"‘ network is loaded.

Value

A list containing PPI and drug graph components.

Degree_centrality *Degree Centrality for Network-Based Drug Searching*

Description

Prioritizes candidate drugs or target nodes using degree centrality on an integrated protein-protein interaction and drug-target network.

Usage

```
Degree_centrality(
  object = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  max_deg = NULL,
  result_size = NULL,
  target = "drug",
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  filter_paths = TRUE,
  force = FALSE,
  auth_token = NULL
)

## S4 method for signature 'NetworkBased'
Degree_centrality(object)
```

Arguments

object	Optional ‘NetworkBased‘ object. If ‘NULL‘, a new object is created from the supplied network inputs and method parameters.
ppi_network	Protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default ‘"gene_gene"‘ network is loaded.

`drug_target_network`
: Drug-target interaction network. Expected to contain columns 'ID', 'Drug', 'Target', and 'Group'. If 'NULL', the default '"drug_target"' network is loaded.

`disease_genes`
: Data frame containing disease seed genes in a column named 'gene'.

`max_deg`
: Maximum allowed node degree. Nodes with degree greater than this value are excluded. If 'NULL', defaults to '.Machine$integer.max'.

`result_size`
: Number of top-ranked results to return. If 'NULL', it is inferred from the number of unique drugs in 'drug_target_network'.

`target`
: Node type to prioritize. Default is '"drug"'.

`include_indirect_drugs`
: Logical; whether to include drugs indirectly connected to disease genes. Default is 'TRUE'.

`include_non_approved_drugs`
: Logical; whether to include non-approved or investigational drugs. Default is 'TRUE'.

`filter_paths`
: Logical; whether to restrict graph paths according to the backend implementation. Default is 'TRUE'.

`force`
: Logical; force redownload of default networks from Synapse instead of using the local cache.

`auth_token`
: Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

Degree centrality is a local network measure that ranks nodes by their number of direct connections. In network-based drug repurposing, it can prioritize drugs or targets connected to the disease module in the integrated interactome.

The integrated network is built from a protein-protein interaction network ('ppi_network') and a drug-target interaction network ('drug_target_network'). Disease genes are used as seed nodes. If either network is 'NULL', the corresponding high-confidence default DrugSigNet network is loaded with 'load_drugsignet_network()' from the local cache or Synapse.

This implementation follows the network-based drug search setting used in CADDIE, where disease seed genes are analyzed on a combined interactome of protein-protein and drug-target interactions.

Value

A 'NetworkBased' object containing degree centrality results in 'object@result' and method parameters in 'object@parameters'.

Functions

- `Degree_centrality(NetworkBased)`: Implements the Degree Centrality calculation for the NetworkCentrality object.

References

Hartung M, Anastasi E, Mamdouh ZM, Nogales C, Schmidt HHHW, Baumbach J, Zolotareva O, List M. Cancer driver drug interaction explorer. *Nucleic Acids Research*. 2022;50(W1):W138-W144. doi:10.1093/nar/gkac384

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

# Run with default DrugSigNet networks
res <- Degree_centrality(
  disease_genes = disease_genes,
  target = "drug"
)

# Run with user-provided networks
ppi_network <- data.frame(
  gene1 = c("ENSG00000130203", "ENSG00000142192"),
  gene2 = c("ENSG00000171867", "ENSG00000198786")
)

drug_target_network <- data.frame(
  ID = c("DB0001", "DB0002"),
  Drug = c("drug_a", "drug_b"),
  Target = c("ENSG00000130203", "ENSG00000171867"),
  Group = c("approved", "approved")
)

res_custom <- Degree_centrality(
  ppi_network = ppi_network,
  drug_target_network = drug_target_network,
  disease_genes = disease_genes,
  target = "drug"
)

## End(Not run)
```

`Diffusion` *Diffusion for Network-Based Drug Searching*

Description

Prioritizes candidate drugs using diffusion-based scoring on an integrated protein-protein interaction and drug-target network.

Usage

```
Diffusion(
  object = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  ties_method = "max",
  output_dir = tempdir(),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'NetworkBased'
Diffusion(object)
```

Arguments

object	Optional 'NetworkBased' object. If 'NULL', a new object is created from the supplied network inputs and method parameters.
ppi_network	Protein-protein interaction network. Expected to contain columns 'gene1' and 'gene2'. If 'NULL', the default '"gene_gene"' network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns 'ID', 'Drug', 'Target', and 'Group'. If 'NULL', the default '"drug_target"' network is loaded.
disease_genes	Data frame containing disease seed genes in a column named 'gene'.
include_indirect_drugs	Logical; whether to include drugs indirectly connected to disease genes. Default is 'TRUE'.
include_non_approved_drugs	Logical; whether to include non-approved or investigational drugs. Default is 'TRUE'.
ties_method	Method used to resolve ties in diffusion-based rankings. Default is '"max"'.
output_dir	Directory used to store diffusion similarity matrices and related intermediate files. Defaults to 'tempdir()'.
force	Logical; force redownload of default networks from Synapse instead of using the local cache.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

Diffusion methods propagate signal from disease seed genes across a network so that nearby nodes receive higher scores. In network-based drug repurposing, this prioritizes drugs whose targets are close to disease-associated genes in the integrated interactome.

The integrated network is built from a protein-protein interaction network (‘ppi_network‘) and a drug-target interaction network (‘drug_target_network‘). Disease genes are used as seed nodes. If either network is ‘NULL‘, the corresponding high-confidence default DrugSigNet network is loaded with ‘load_drugsignet_network()‘ from the local cache or Synapse.

Diffusion intermediate files are stored in ‘output_dir‘. If the required similarity matrices are missing, they are generated automatically and reused in later analyses that use the same directory.

This implementation follows the network medicine framework used to identify drug-repurposing opportunities for COVID-19.

Value

A ‘NetworkBased‘ object containing diffusion results in ‘object@result‘ and method parameters in ‘object@parameters‘. The resolved output directory is stored in ‘object@parameters$output_dir‘.

References

Gysi DM, do Valle Í, Zitnik M, Ameli A, Gan X, Varol O, Ghiassian SD, Patten JJ, Davey RA, Loscalzo J, Barabási AL. Network medicine framework for identifying drug-repurposing opportunities for COVID-19. *Proceedings of the National Academy of Sciences of the United States of America*. 2021;118(19):e2025581118. doi:10.1073/pnas.2025581118

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

# Run with default DrugSigNet networks
res <- Diffusion(
  disease_genes = disease_genes,
  output_dir = tempdir()
)

# Run with user-provided networks
ppi_network <- data.frame(
  gene1 = c("ENSG00000130203", "ENSG00000142192"),
  gene2 = c("ENSG00000171867", "ENSG00000198786")
)

drug_target_network <- data.frame(
  ID = c("DB0001", "DB0002"),
  Drug = c("drug_a", "drug_b"),
  Target = c("ENSG00000130203", "ENSG00000171867"),
  Group = c("approved", "approved")
)

res_custom <- Diffusion(
  ppi_network = ppi_network,
  drug_target_network = drug_target_network,
  disease_genes = disease_genes,
```

```
  ties_method = "max",
  output_dir = tempdir()
)

## End(Not run)
```

`disease_seeds` *Disease Seed Genes*

Description

An example set of disease-associated seed genes for demonstrating network-based drug repurposing with DrugSigNet.

Usage

```
disease_seeds
```

Format

A data frame with one column:

gene Ensembl gene identifier of a SARS-CoV-2 host protein target.

Details

The seed genes are derived from the SARS-CoV-2 host protein targets used by Gysi et al. (2021) to define the COVID-19 disease module for network-based drug repurposing. The original set comprised 332 experimentally identified human proteins targeted by SARS-CoV-2 proteins. Gene symbols were mapped to Ensembl gene identifiers for use with the DrugSigNet network-based workflow.

Source

Gysi DM, do Valle I, Zitnik M, et al. (2021). Network medicine framework for identifying drug-repurposing opportunities for COVID-19. *Proceedings of the National Academy of Sciences*, 118(19), e2025581118. doi:10.1073/pnas.2025581118

Examples

```
data(disease_seeds)
head(disease_seeds)
```

`disease_signature` *Disease Gene-Expression Signature*

Description

An example disease gene-expression signature for demonstrating signature-based drug repurposing with DrugSigNet.

Usage

```
disease_signature
```

Format

A data frame with two columns:

Entrez NCBI Entrez Gene identifier.

FC Log2 fold-change for COVID-19 convalescent donors relative to healthy donors. Positive and negative values indicate higher and lower expression in convalescent donors, respectively.

Details

The signature is derived from peripheral blood leukocyte transcriptomic profiles of COVID-19 convalescent donors (CCD) compared with healthy donors (HD), as reported by Gedda et al. (2022). Genes from the 120–149 day post-symptom-onset comparison were retained when the false discovery rate (FDR) was less than or equal to 0.05 and a log2 fold-change value was available. Gene identifiers were mapped to NCBI Entrez Gene identifiers.

Source

Gedda MR, Danaher P, Shao L, et al. (2022). Longitudinal transcriptional analysis of peripheral blood leukocytes in COVID-19 convalescent donors. *Journal of Translational Medicine*, 20, 587. doi:10.1186/s12967022037517

Examples

```
data(disease_signature)
head(disease_signature)
```

`Dowdall` *Dowdall Rank Aggregation Method*

Description

Aggregates multiple ranked drug lists using the Dowdall method.

Usage

```
Dowdall(object = NULL, input_data, ties_method = "max", reverse = FALSE)
```

Arguments

`object`	Optional 'RankAggregation' object. For pipe-friendly use, this can also be a data frame used as 'input_data'.
`input_data`	Data frame containing items and ranking scores. The first column should contain item identifiers, and at least two additional columns must be numeric.
`ties_method`	Method used to resolve ties during rank conversion. One of '"max"', '"min"', '"average"', '"first"', '"last"', '"random"', or '"dense"'. Default is '"max"'.
`reverse`	Logical; if 'FALSE', smaller input values are treated as better, which is appropriate when numeric columns already represent ranks. If 'TRUE', larger input values are treated as better. Default is 'FALSE'.

Details

'Dowdall()' combines rankings from at least two numeric columns in 'input_data'. The first column is treated as the item identifier, typically 'Drug', and the remaining numeric columns are converted to ranks before aggregation.

The Dowdall method is a positional voting system that assigns each item a reciprocal rank score ('1 / rank') within each input ranking. These reciprocal scores are summed across ranking columns to produce a consensus score. Items ranked highly across several methods receive larger Dowdall scores and better final consensus ranks.

In DrugSigNet, reciprocal rank scores are scaled to the range '[0, 1]' before summation so that different ranking methods contribute comparably.

Missing numeric values are replaced with zero before ranking. Values equal to zero are treated as unranked entries and contribute zero to the final consensus score.

For pipe-friendly use, a data frame can be supplied as 'object'; in that case, it is used as 'input_data'.

Value

A 'RankAggregation' object containing the Dowdall aggregation results in 'object@result'.

References

Reilly B. Social choice in the south seas: Electoral innovation and the Borda count in the Pacific Island countries. *International Political Science Review*. 2002;23(4):355-372. doi:10.1177/0192512102023004002

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3)
)

res <- Dowdall(
  input_data = rank_df,
  ties_method = "max",
  reverse = FALSE
)

# Pipe-friendly use
res_pipe <- Dowdall(rank_df, reverse = FALSE)

## End(Not run)
```

Dowdall,RankAggregation-method
Dowdall for a RankAggregation Object

Description

Dowdall for a RankAggregation Object

Usage

```
## S4 method for signature 'RankAggregation'
Dowdall(object)
```

Arguments

`object`	A `RankAggregation` object containing the input rankings and Dowdall parameters.

Value

The supplied `RankAggregation` object with aggregation results stored in its `result` slot.

drugNetworkPipeline *Drug Network-Based Pipeline*

Description

Runs a complete network-based drug prioritization workflow using centrality, diffusion, network proximity, rank aggregation, and optional drug annotation.

Usage

```
drugNetworkPipeline(
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  run_all_network_methods = TRUE,
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  hub_penalty = 0.01,
  damping_factor = 0.95,
  result_size = NULL,
  n_simulations = 1000,
  n_workers = 5,
  prior = 0.093,
  num_bin = 200,
  ties_method = "max",
  output_dir = NULL,
  top_k = 100,
  trial_condition = NULL,
  drug_annotation_source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  target_id_from = NULL,
  force = FALSE,
  auth_token = NULL,
  run_drug_annotation = TRUE,
  run_visualization = TRUE
)
```

Arguments

ppi_network	Optional protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default DrugSigNet gene-gene network is loaded.
drug_target_network	Optional drug-target interaction network. Expected to contain columns such as ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default DrugSigNet drug-target network is loaded.
disease_genes	Disease-associated genes used as seed genes. Can be a character vector or a data frame containing a ‘gene‘ column.

`run_all_network_methods`
: Logical; if 'TRUE', run TrustRank, harmonic centrality, degree centrality, network proximity, and diffusion. If 'FALSE', run only TrustRank, harmonic centrality, and degree centrality. Default is 'TRUE'.

`include_indirect_drugs`
: Logical; whether to include drugs indirectly connected to disease genes. Default is 'TRUE'.

`include_non_approved_drugs`
: Logical; whether to include non-approved or investigational drugs. Default is 'TRUE'.

`hub_penalty`
: Numeric hub penalty used by centrality-based methods. If a single value is provided, it is used for both TrustRank and harmonic centrality. If two values are provided, the first is used for TrustRank and the second for harmonic centrality. Default is '0.01'.

`damping_factor`
: Numeric damping factor used by TrustRank. Must be between '0' and '1'. Default is '0.95'.

`result_size`
: Optional number of ranked results returned per method. If 'NULL', it is inferred from the number of unique drugs in the resolved drug-target network.

`n_simulations`
: Number of random simulations used by 'Network_proximity()'. Default is '1000'.

`n_workers`
: Number of workers used by methods that support parallel execution, currently network proximity. Centrality methods are run sequentially for reticulate/Python backend stability. Default is '5'.

`prior`
: Numeric prior used by CRank aggregation. Default is '0.093'.

`num_bin`
: Number of bins used by CRank aggregation. Default is '200'.

`ties_method`
: Method used to resolve ranking ties. One of '"max"', '"min"', '"average"', or '"dense"'. Default is '"max"'.

`output_dir`
: Optional directory used by diffusion methods to store intermediate files.

`top_k`
: Number of top-ranked drugs used for downstream annotation and target enrichment. Default is '100'.

`trial_condition`
: Optional condition used to retrieve matching clinical trial annotations.

`drug_annotation_source`
: Drug annotation source. One of '"All"', '"OpenTargets"', '"CHEMBL"', or '"TTD"'.

`target_id_from`
: Optional AnnotationDbi keytype or supported alias describing target identifiers used for target set enrichment. If 'NULL', targets are treated as HGNC symbols unless Ensembl gene IDs are detected.

`force`
: Logical; force refresh of Synapse-backed annotation or network resources where supported. Default is 'FALSE'.

`auth_token`
: Optional Synapse authentication token used by annotation and network-loading helpers.

`run_drug_annotation`
: Logical; if 'TRUE', annotate ranked drugs and run target set enrichment. Default is 'TRUE'.

`run_visualization`
: Logical; if 'TRUE', build visualization metadata and plots. Default is 'TRUE'.

Details

'drugNetworkPipeline()' prioritizes candidate drugs from disease-associated genes using network-based methods. The workflow first resolves the protein-protein interaction network ('ppi_network') and drug-target interaction network ('drug_target_network'). If either network is 'NULL', the corresponding default DrugSigNet network can be loaded from the local cache or Synapse.

The pipeline always runs centrality-based methods, including TrustRank, harmonic centrality, and degree centrality. If 'run_all_network_methods = TRUE', it additionally runs network proximity and diffusion-based methods.

Method-specific rankings are integrated using CRank, Dowdall, and Robust Rank Aggregation (RRA). The final harmonized network ranking is stored in the 'Network_Harmonized' section of the returned object.

If 'run_drug_annotation = TRUE', ranked drugs are annotated and target set enrichment analysis is performed on targets of top-ranked drugs. If 'run_visualization = TRUE', visualization-ready outputs are included in the returned pipeline object.

Value

A 'DrugSearchingPipeline' S4 object containing raw network-search results, processed rankings, rank aggregation results, and optionally annotation and visualization sections.

See Also

'TrustRank()', 'Harmonic_centrality()', 'Degree_centrality()', 'Network_proximity()', 'Diffusion()', 'CRank()', 'Dowdall()', 'RRA()', 'annotate_drugs()', 'TSEA()'

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

# Run network workflow using default DrugSigNet networks
res <- drugNetworkPipeline(
  disease_genes = disease_genes,
  top_k = 100
)

# Run only centrality-based network methods
res_centrality <- drugNetworkPipeline(
  disease_genes = disease_genes,
  run_all_network_methods = FALSE,
  top_k = 100
)

# Run with user-provided networks
ppi_network <- data.frame(
  gene1 = c("ENSG00000130203", "ENSG00000142192"),
  gene2 = c("ENSG00000171867", "ENSG00000198786")
)
```

```
drug_target_network <- data.frame(
  ID = c("DB0001", "DB0002"),
  Drug = c("drug_a", "drug_b"),
  Target = c("ENSG00000130203", "ENSG00000171867"),
  Group = c("approved", "approved")
)

res_custom <- drugNetworkPipeline(
  ppi_network = ppi_network,
  drug_target_network = drug_target_network,
  disease_genes = disease_genes,
  run_all_network_methods = FALSE,
  top_k = 100
)

## End(Not run)
```

---

drugRepurposingPipeline

Run Unified Drug Repurposing Workflow

Description

Runs signature-based, network-based, or integrated drug repurposing workflows in a single call.

Usage

```
drugRepurposingPipeline(
  signature_input = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes = NULL,
  mode = c("signature", "network", "both"),
  padj = 0.05,
  trend = NULL,
  drug_name_synonym = NULL,
  ties_method = "max",
  prior = 0.093,
  num_bin = 200,
  n_workers = 1,
  chunk_size = 5000,
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  signature_refdb_auth_token = NULL,
  validate_signature_refdb = TRUE,
  top_k = 100,
  trial_condition = NULL,
```

```
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  run_all_network_methods = TRUE,
  hub_penalty = 0.01,
  damping_factor = 0.95,
  result_size = NULL,
  n_simulations = 1000,
  output_dir = NULL,
  drug_annotation_source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  target_id_from = NULL,
  force = FALSE,
  auth_token = NULL,
  run_drug_annotation = TRUE,
  run_visualization = TRUE
)
```

Arguments

signature_input	Data frame with columns ‘Entrez‘ and ‘FC‘.
ppi_network	Optional protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default DrugSigNet gene-gene network is loaded.
drug_target_network	Optional drug-target interaction network. Expected to contain columns such as ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default DrugSigNet drug-target network is loaded.
disease_genes	Disease-associated genes used as seed genes. Can be a character vector or a data frame containing a ‘gene‘ column.
mode	Workflow mode. One of ‘"signature"‘, ‘"network"‘, or ‘"both"‘.
padj	Optional adjusted p-value threshold passed to supported signature-search methods. Default is ‘0.05‘.
trend	Optional character filter for perturbation direction. Use ‘"up"‘, ‘"down"‘, or ‘NULL‘. Default is ‘NULL‘.
drug_name_synonym	Optional drug synonym table used to harmonize drug names across signature databases. Must contain ‘ID‘, ‘Drug‘, ‘name_synonym_list‘, and ‘Group‘ columns. Legacy lowercase ‘name‘ and ‘group‘ column names are also accepted.
ties_method	Method used to resolve ranking ties. Common options are ‘"max"‘, ‘"min"‘, and ‘"dense"‘. Default is ‘"max"‘.
prior	Numeric prior used by CRank aggregation. Default is ‘0.093‘.
num_bin	Number of bins used by CRank aggregation. Default is ‘200‘.
n_workers	Number of parallel workers used by supported pipeline components.
chunk_size	Integer; number of reference signatures processed per chunk by signature-search methods. Default is ‘5000‘.

signature_refdb_mode
: Signature reference database mode. Use '"default"' for the default ExperimentHub reference, '"frozen"' for the cached Synapse HDF5 reference database, or '"frozen_force"' to redownload the frozen database.

signature_refdb_auth_token
: Optional Synapse authentication token used for frozen signature reference databases. If 'NULL', 'auth_token' or the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

validate_signature_refdb
: Logical; whether to validate frozen HDF5 reference databases before use. Default is 'TRUE'.

top_k
: Number of top-ranked drugs used for downstream annotation and target enrichment. Default is '100'.

trial_condition
: Optional condition used to retrieve matching clinical trial annotations. Default is 'NULL'.

include_indirect_drugs
: Logical; whether to include drugs indirectly connected to disease genes. Default is 'TRUE'.

include_non_approved_drugs
: Logical; whether to include non-approved or investigational drugs. Default is 'TRUE'.

run_all_network_methods
: Logical; if 'TRUE', run TrustRank, harmonic centrality, degree centrality, network proximity, and diffusion. If 'FALSE', run only TrustRank, harmonic centrality, and degree centrality. Default is 'TRUE'.

hub_penalty
: Numeric hub penalty used by centrality-based methods. If a single value is provided, it is used for both TrustRank and harmonic centrality. If two values are provided, the first is used for TrustRank and the second for harmonic centrality. Default is '0.01'.

damping_factor
: Numeric damping factor used by TrustRank. Must be between '0' and '1'. Default is '0.95'.

result_size
: Optional number of ranked results returned per method. If 'NULL', it is inferred from the number of unique drugs in the resolved drug-target network.

n_simulations
: Number of random simulations used by 'Network_proximity()'. Default is '1000'.

output_dir
: Optional directory used by diffusion methods to store intermediate files.

drug_annotation_source
: Drug annotation source used by network and integrated workflows. One of '"All"', '"OpenTargets"', '"CHEMBL"', or '"TTD"'.

target_id_from
: Optional AnnotationDbi keytype or supported alias describing target identifiers used for enrichment. If 'NULL', targets are treated as HGNC symbols unless Ensembl gene IDs are detected.

force
: Logical; force refresh of Synapse-backed annotation or reference resources where supported. Default is 'FALSE'.

auth_token
: Optional Synapse authentication token used by annotation and reference database helpers.

`run_drug_annotation`
: Logical; if 'TRUE', annotate ranked drugs and run target set enrichment. Default is 'TRUE'.

`run_visualization`
: Logical; if 'TRUE', build visualization metadata and plots. Default is 'TRUE'.

Details

'drugRepurposingPipeline()' provides a unified interface to the main DrugSigNet workflows. Depending on 'mode', it runs 'drugSignaturePipeline()', 'drugNetworkPipeline()', or both followed by 'integrateSignatureNetwork()'.

In 'mode = "signature"', only signature-based drug searching is performed. In 'mode = "network"', only network-based drug searching is performed. In 'mode = "both"', the signature and network workflows are run separately and then integrated into a combined result.

Signature mode uses 'padj = 0.05' by default and forwards it directly to 'drugSignaturePipeline()'. To reproduce the same results with a standalone call, explicitly pass 'padj = 0.05' there as well. Use the exact argument name 'n_workers' in both interfaces.

If 'run_drug_annotation = TRUE', downstream annotation and target set enrichment are performed where supported. If 'run_visualization = TRUE', visualization-ready outputs are included in the returned object.

Value

A 'DrugSearchingPipeline' S4 object containing workflow-specific raw results, processed rankings, rank aggregation results, and optionally annotation and visualization sections.

Examples

```
## Not run:
signature_input <- data.frame(
  Entrez = c("7157", "1956", "5290", "7422"),
  FC = c(1.35, -0.82, 2.11, -1.47)
)

disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

res_signature <- drugRepurposingPipeline(
  signature_input = signature_input,
  mode = "signature",
  top_k = 100
)

res_network <- drugRepurposingPipeline(
  disease_genes = disease_genes,
  mode = "network",
  top_k = 100
)
```

```
res_both <- drugRepurposingPipeline(
  signature_input = signature_input,
  disease_genes = disease_genes,
  mode = "both",
  top_k = 100,
  n_workers = 1
)

## End(Not run)
```

`drugs10` *Example Drug Names*

Description

Ten example perturbagen names for DrugSigNet demonstrations.

Format

A character vector of length 10.

`DrugSearchingPipeline` *Construct a DrugSearchingPipeline object*

Description

Construct a DrugSearchingPipeline object

Usage

```
DrugSearchingPipeline(
  DrugSearching = list(Raw = list(), Processed = list()),
  RankAggregation = list(),
  DrugAnnotation = NULL,
  Visualization = NULL,
  type = "signature"
)
```

Arguments

`DrugSearching`	List containing ‘Raw‘ and ‘Processed‘ entries.
`RankAggregation`	List of rank-aggregation results.
`DrugAnnotation`	Optional list of annotation results, or NULL to omit the annotation slot.
`Visualization`	Optional list of visualization outputs, or NULL to omit the visualization slot.
`type`	Pipeline type label (for example ‘"signature"‘ or ‘"network"‘).

Value

A valid pipeline S4 object whose slots follow the order 'DrugSearching', 'RankAggregation', optional 'DrugAnnotation', optional 'Visualization', and 'type'.

`DrugSearchingPipeline-class`
DrugSearchingPipeline Class Definition

Description

A class to manage drug signature searching and processing.

`DrugSearchingPipelineAnnotated-class`
Annotated Drug-Searching Pipeline Class

Description

Stores raw and processed searches, rank aggregations, and drug annotations.

Slots

`DrugSearching` A list containing `Raw` and `Processed` search results.

`RankAggregation` A list of rank-aggregation results.

`DrugAnnotation` A list of drug-annotation results.

`type` A character string identifying the pipeline type.

`DrugSearchingPipelineAnnotatedVisualized-class`
Annotated and Visualized Drug-Searching Pipeline Class

Description

Stores search, rank-aggregation, annotation, and visualization results.

Slots

`DrugSearching` A list containing `Raw` and `Processed` search results.

`RankAggregation` A list of rank-aggregation results.

`DrugAnnotation` A list of drug-annotation results.

`Visualization` A list of visualization results.

`type` A character string identifying the pipeline type.

DrugSearchingPipelineVisualized-class *Visualized Drug-Searching Pipeline Class*

Description

Stores raw and processed searches, rank aggregations, and visualizations.

Slots

`DrugSearching` A list containing `Raw` and `Processed` search results.

`RankAggregation` A list of rank-aggregation results.

`Visualization` A list of visualization results.

`type` A character string identifying the pipeline type.

drugSignaturePipeline *Drug Signature-Based Pipeline*

Description

Runs a complete signature-based drug repurposing workflow from differential expression input.

Usage

```
drugSignaturePipeline(
  signature_input,
  padj = NULL,
  trend = NULL,
  drug_name_synonym = NULL,
  ties_method = "max",
  prior = 0.093,
  num_bin = 200,
  n_workers = 1,
  chunk_size = 5000,
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  signature_refdb_auth_token = NULL,
  validate_signature_refdb = TRUE,
  top_k = 100,
  trial_condition = NULL,
  target_id_from = NULL,
  force = FALSE,
  auth_token = NULL,
  run_drug_annotation = TRUE,
  run_visualization = TRUE
)
```

Arguments

signature_input	Data frame with columns ‘Entrez‘ and ‘FC‘.
padj	Optional adjusted p-value threshold passed to supported signature-search methods. Default is ‘NULL‘.
trend	Optional character filter for perturbation direction. Use ‘"up"‘, ‘"down"‘, or ‘NULL‘. Default is ‘NULL‘.
drug_name_synonym	Optional drug synonym table used to harmonize drug names across signature databases. Must contain ‘ID‘, ‘Drug‘, ‘name_synonym_list‘, and ‘Group‘ columns. Legacy lowercase ‘name‘ and ‘group‘ column names are also accepted.
ties_method	Method used to resolve ranking ties. Common options are ‘"max"‘, ‘"min"‘, and ‘"dense"‘. Default is ‘"max"‘.
prior	Numeric prior used by CRank aggregation. Default is ‘0.093‘.
num_bin	Number of bins used by CRank aggregation. Default is ‘200‘.
n_workers	Number of parallel workers used for signature-search methods. Default is ‘1‘.
chunk_size	Integer; number of reference signatures processed per chunk by each signature-search method. Default is ‘5000‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.
signature_refdb_auth_token	Optional Synapse authentication token used when downloading frozen signature reference databases. If ‘NULL‘, ‘auth_token‘ or the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.
validate_signature_refdb	Logical; whether to validate frozen HDF5 reference databases before use. Default is ‘TRUE‘.
top_k	Number of top-ranked drugs used for downstream annotation and target enrichment. Default is ‘100‘.
trial_condition	Optional condition used to retrieve matching clinical trial annotations. Default is ‘NULL‘.
target_id_from	Optional AnnotationDbi keytype or supported alias describing target identifiers used for enrichment. If ‘NULL‘, targets are treated as HGNC symbols unless Ensembl gene IDs are detected.
force	Logical; force refresh of Synapse-backed annotation or reference resources where supported. Default is ‘FALSE‘.
auth_token	Optional Synapse authentication token used by annotation and reference database helpers.
run_drug_annotation	Logical; if ‘TRUE‘, annotate ranked drugs and run target set enrichment. Default is ‘TRUE‘.
run_visualization	Logical; if ‘TRUE‘, build visualization metadata and plots. Default is ‘TRUE‘.

Details

'drugSignaturePipeline()' performs signature-based drug searching using multiple methods, including CMAP, LINCS, gCMAP, and correlation-based search. The resulting method-specific drug rankings are harmonized and integrated using rank aggregation methods.

The input 'signature_input' should contain two columns: 'Entrez', containing Entrez gene identifiers, and 'FC', containing fold-change values, typically log2 fold changes. Positive and negative fold changes are used to define up- and down-regulated gene sets.

Drug names can optionally be harmonized using 'drug_name_synonym'. When provided, the synonym table is standardized internally and used to map perturbation names across reference databases.

Frozen signature reference databases can be used by setting 'signature_refdb_mode = "frozen"' or '"frozen_force"'. In this case, reference databases are downloaded through 'load_signature_refdb()' and cached locally using the supplied Synapse authentication token.

If 'run_drug_annotation = TRUE', top-ranked drugs are annotated and target set enrichment analysis is performed using targets of the selected 'top_k' drugs. If 'run_visualization = TRUE', visualization-ready outputs are added to the returned pipeline object.

Value

A 'DrugSearchingPipeline' S4 object containing raw signature-search results, processed rankings, rank aggregation results, and optionally drug annotation and visualization sections.

Examples

```
## Not run:
signature_input <- data.frame(
  Entrez = c("7157", "1956", "5290", "7422"),
  FC = c(1.35, -0.82, 2.11, -1.47)
)

res <- drugSignaturePipeline(
  signature_input = signature_input,
  top_k = 100,
  n_workers = 1
)

# Use frozen Synapse reference databases
res_frozen <- drugSignaturePipeline(
  signature_input = signature_input,
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN"),
  top_k = 100
)

## End(Not run)
```

Drug_target_reformat *Reformat Drug-Target Network by Target Set*

Description

Groups drugs that share identical target sets and returns a normalized drug-target table compatible with network-based methods.

Usage

```
Drug_target_reformat(df)
```

Arguments

df	A data frame containing at least 'ID', 'Drug', 'Target', and 'Group'.

Value

A tibble with columns 'ID', 'Drug', 'Target', 'Group', 'Target_Group_ID'.

extract_drug_subgraph *Extract a Drug-Centered PPI Subgraph*

Description

Extracts a drug-centered protein-protein interaction subgraph from drug-target interactions, PPI edges, disease genes, and optionally drug-induced signature genes.

Usage

```
extract_drug_subgraph(
  ppi = NULL,
  dti = NULL,
  disease_genes_df = NULL,
  drug_signature_df = NULL,
  drug = NULL,
  k_hops = 2,
  drug_network = drug,
  drug_signature_id = drug,
  k2_filter_by = c("none", "disease", "drug_signature"),
  compute_drug_signature = FALSE,
  Gene_map = NULL,
  refdb = c("cmap", "lincs2"),
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  signature_cells = NULL,
  signature_profile_ids = NULL,
```

```
  n_up = 100,
  n_down = 100,
  force = FALSE,
  auth_token = NULL
)
```

Arguments

ppi	Optional PPI network data frame with ‘gene1‘ and ‘gene2‘ columns. If ‘NULL‘, the default DrugSigNet gene-gene network is loaded.
dti	Optional drug-target interaction data frame with ‘Drug‘ and ‘Target‘ columns. If ‘NULL‘, the default DrugSigNet drug-target network is loaded.
disease_genes_df	Data frame with ‘Ensembl‘ and ‘source‘ columns.
drug_signature_df	Optional data frame with an ‘Ensembl‘ column.
drug	Character string specifying the drug name or identifier.
k_hops	Positive integer number of hops to extract. Default is ‘2‘.
drug_network	Drug label used in the drug-target/network table. Defaults to ‘drug‘.
drug_signature_id	Drug label used when computing a drug signature. Defaults to ‘drug‘.
k2_filter_by	Node set used for second and later PPI hops. One of ‘"none"‘, ‘"disease"‘, or ‘"drug_signature"‘. Default is ‘"none"‘.
compute_drug_signature	Logical; if ‘TRUE‘, compute ‘drug_signature_df‘ with ‘get_drug_signature()‘. Default is ‘FALSE‘.
Gene_map	Optional gene mapping table passed to ‘get_drug_signature()‘ when ‘compute_drug_signature = TRUE‘.
refdb	Reference database. One of ‘"cmap"‘ or ‘"lincs2"‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.
signature_cells	Optional character vector of cell lines passed to ‘get_drug_signature()‘ when computing the drug signature. For CMAP, this can reproduce a cell-restricted legacy analysis.
signature_profile_ids	Optional character vector of exact reference profile column names passed to ‘get_drug_signature()‘. This is especially useful for reproducing exact LINCS2 ‘pert_id__cell__type‘ selections.
n_up	Optional positive integer specifying the number of top up-regulated genes to return. If ‘NULL‘, no separate up-regulated gene table is returned.
n_down	Optional positive integer specifying the number of top down-regulated genes to return. If ‘NULL‘, no separate down-regulated gene table is returned.

force	Logical; if 'TRUE', force refresh of default network cache when 'ppi' or 'dti' is 'NULL'. Default is 'FALSE'.
auth_token	Optional Synapse authentication token used when loading default networks or frozen signature reference databases.

Details

'extract_drug_subgraph()' builds a local network around a selected drug. The first hop connects the drug to its direct targets in the drug-target interaction table. Additional hops expand through the supplied PPI network.

If 'ppi' or 'dti' is 'NULL', the corresponding default DrugSigNet network is loaded from the local cache or Synapse using 'load_drugsignet_network()'. Default networks are filtered to high-confidence PPI edges and high-confidence drug-target interactions with high- or medium-confidence targets when these confidence columns are available.

Expansion after the first hop can be restricted with 'k2_filter_by'. Use '"none"' to traverse all PPI nodes, '"disease"' to restrict traversal to disease genes, or '"drug_signature"' to restrict traversal to genes in the supplied or computed drug signature.

When 'compute_drug_signature = TRUE', drug-induced signature genes are computed with 'get_drug_signature()' using the selected 'refdb', 'signature_refdb_mode', 'n_up', and 'n_down' settings. Optional 'signature_cells' or 'signature_profile_ids' can restrict the reference profiles so that a predefined or legacy analysis can be reproduced.

Value

A list with two elements:

subgraph Edge data frame for the extracted subgraph.

metadata List containing drug labels, optional drug signature data, hop settings, reference database settings, and whether default networks were loaded.

Returns 'NULL' if no valid graph can be built or if the selected drug is not present in the resolved network.

Examples

```
## Not run:
subg <- extract_drug_subgraph(
  disease_genes_df = disease_genes,
  drug = "miglitol",
  k_hops = 2
)

subg_sig <- extract_drug_subgraph(
  disease_genes_df = disease_genes,
  drug = "miglitol",
  compute_drug_signature = TRUE,
  refdb = "lincs2",
  n_up = 100,
  n_down = 100,
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
```

```
)

## End(Not run)
```

`extract_top_ranked_drugs` *Extract Top-Ranked Drugs Across Rank Columns*

Description

Extracts the union of drugs ranked within the top ‘top_n‘ in one or more selected rank columns.

Usage

```
extract_top_ranked_drugs(df, rank_cols = NULL, top_n = 100, id_col = "Drug")

get_top_union_drugs(df, rank_cols, top_n = 100, id_col = "Drug")
```

Arguments

`df`	Data frame containing a drug identifier column and one or more numeric rank columns.
`rank_cols`	Character vector of rank columns to evaluate. Missing columns are ignored. If ‘NULL‘, all numeric columns except ‘id_col‘ are used.
`top_n`	Positive integer number of top-ranked entries to keep per rank column. Default is ‘100‘.
`id_col`	Name of the drug identifier column. Default is ‘"Drug"‘.

Details

‘extract_top_ranked_drugs()‘ is used to collect drugs that appear among the top-ranked candidates in at least one ranking method. A drug is retained if any selected rank column is less than or equal to ‘top_n‘.

If multiple rows are present for the same drug, the function returns one row per drug and keeps the minimum rank observed for each selected rank column. Missing rank columns are ignored. If ‘rank_cols = NULL‘, all numeric columns except the identifier column are used.

‘get_top_union_drugs()‘ is a convenience wrapper that returns only the unique drug identifiers from ‘extract_top_ranked_drugs()‘.

Value

A data frame containing one row per selected drug, the identifier column, and the selected rank columns. `get_top_union_drugs()` returns a one-column data frame containing the unique selected drug identifiers.

Examples

```
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c", "drug_a"),
  Method1 = c(1, 50, 120, 3),
  Method2 = c(150, 2, 80, 10)
)

extract_top_ranked_drugs(
  df = rank_df,
  rank_cols = c("Method1", "Method2"),
  top_n = 100
)

get_top_union_drugs(
  df = rank_df,
  rank_cols = c("Method1", "Method2"),
  top_n = 100
)
```

---

gcmap_method *gCMAP Method for Signature-Based Drug Searching*

Description

Performs gene expression signature-based drug repurposing using the gCMAP algorithm.

Usage

```
gcmap_method(
  object = NULL,
  signature_matrix,
  ref_db,
  higher = NULL,
  lower = NULL,
  padj = NULL,
  chunk_size = 5000,
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  auth_token = NULL,
  validate_signature_refdb = TRUE
)

## S4 method for signature 'SignatureBased'
gcmap_method(object)
```

Arguments

object	Optional 'SignatureBased' object. If 'NULL', a new object is created from 'signature_matrix', 'ref_db', 'higher', 'lower', and 'padj'.
signature_matrix	Numeric matrix containing log2 fold changes. The matrix must have one column, with gene identifiers stored as row names.
ref_db	Reference database. Use '"cmap"' or '"lincs2"', or provide a local HDF5 path returned by 'load_signature_refdb()' to use a frozen Synapse reference database.
higher	Numeric threshold for selecting higher-expressed genes. If 'NULL', no upper threshold is applied by this wrapper.
lower	Numeric threshold for selecting lower-expressed genes. If 'NULL', no lower threshold is applied by this wrapper.
padj	Optional adjusted p-value threshold used to filter results when supported by the selected reference database.
chunk_size	Integer; number of reference signatures processed per chunk. Default is '5000'.
signature_refdb_mode	Signature reference database mode. Use '"default"' for the default ExperimentHub reference, '"frozen"' for the cached Synapse HDF5 reference database, or '"frozen_force"' to redownload the frozen database.
auth_token	Optional Synapse authentication token used when 'signature_refdb_mode' requests a frozen database. If 'NULL', 'SYNAPSE_AUTH_TOKEN' is used.
validate_signature_refdb	Logical; whether to validate frozen HDF5 reference databases before use. Default is 'TRUE'.

Details

The method compares a ranked gene expression signature with signatures in a reference database using the gCMAP algorithm. The input 'signature_matrix' must be a numeric matrix containing log2 fold changes, with one column and gene identifiers stored as row names.

'higher' and 'lower' define the thresholds used to select up- and down-regulated genes from the input signature. 'padj' can be used to filter the returned results by adjusted p-value when supported by the reference database.

'ref_db' can be '"cmap"' or '"lincs2"'. A local HDF5 path returned by 'load_signature_refdb()' can also be supplied to use a frozen Synapse reference database. Frozen databases can also be fetched directly by setting 'signature_refdb_mode = "frozen"' or '"frozen_force"' and providing 'auth_token'.

Value

A 'SignatureBased' object containing the gCMAP search results and query metadata.

Functions

- `gcmap_method(SignatureBased)`: Implements the gCMAP method for 'SignatureBased'.

Examples

```
## Not run:
# Differential gene expression signature (log2 fold changes)
signature_matrix <- matrix(
  c(1.35, -0.82, 2.11, -1.47),
  ncol = 1,
  dimnames = list(
    c("7157", "1956", "5290", "7422"),
    "log2FC"
  )
)

# Search using the default CMAP reference database
res <- gcmap_method(
  signature_matrix = signature_matrix,
  ref_db = "cmap",
  higher = 1,
  lower = -1,
  padj = 0.05
)

# Automatically fetch and use a frozen Synapse reference database
res_frozen <- gcmap_method(
  signature_matrix = signature_matrix,
  ref_db = "cmap",
  higher = 1,
  lower = -1,
  padj = 0.05,
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Reuse a local frozen HDF5 reference database
cmap_ref <- load_signature_refdb(
  ref_db = "cmap",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_local <- gcmap_method(
  signature_matrix = signature_matrix,
  ref_db = cmap_ref,
  higher = 1,
  lower = -1,
  padj = 0.05
)

## End(Not run)
```

gcmap_rank_score *Rank gCMAP Signature Results*

Description

Converts raw results from [gcmap_method()] into the standardized ranking format used by the signature pipeline.

Usage

```
gcmap_rank_score(
  gcmap_signature,
  cmap_signature,
  trend_name = NULL,
  ties_method = "max"
)
```

Arguments

gcmap_signature	A data frame of raw gCMAP signature results.
cmap_signature	Corresponding raw CMAP results used to derive the cutoff.
trend_name	Optional trend used to derive the CMAP cutoff.
ties_method	Character string passed to [base::rank()], or ‘"dense"‘.

Value

A data frame of standardized gCMAP rank scores.

generateGraphFiles *Generate Graph Files*

Description

Generates the necessary files for running the TrustRank algorithm.

Usage

```
generateGraphFiles(temp_dir, ppi_network, drug_target_network, disease_genes)
```

Arguments

temp_dir	Temporary directory for storing intermediate files.
ppi_network	Protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default ‘"gene_gene"‘ network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default ‘"drug_target"‘ network is loaded.
disease_genes	Data frame containing disease seed genes in a column named ‘gene‘.

Value

A list of file paths for the graph components.

---

get_drug_adverse_events

Extract Drug Adverse Events and Toxicity Information

Description

Retrieves adverse event, warning, and toxicity annotations for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_adverse_events(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_adverse_events(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
source	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, or ‘"CHEMBL"‘. Default is ‘"All"‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

'get_drug_adverse_events()' extracts safety-related annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a 'DrugAnnotation' object with adverse event and toxicity information stored in 'object@result'.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using 'force' and 'auth_token'.

Supported sources are '"All"', '"OpenTargets"', and '"CHEMBL"'. Returned annotations may include regulatory warning categories and toxicity classes, depending on the selected source.

Not all drugs have available safety annotations; missing values are retained to reflect source data completeness.

Value

A 'DrugAnnotation' object containing a data frame of adverse event, warning, and toxicity annotations in 'object@result'.

Examples

```
## Not run:
res_opentargets <- get_drug_adverse_events(
  drugs = c("Cetirizine", "Pentazocine"),
  source = "OpenTargets",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Phenylbutazone", "Diclofenac sodium")
)

res_chembl <- get_drug_adverse_events(
  drugs = drug_df,
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_all <- get_drug_adverse_events(
  drugs = drug_df,
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

get_drug_atc *Extract Drug ATC Classifications*

Description

Retrieves Anatomical Therapeutic Chemical (ATC) classification annotations for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_atc(
  object = NULL,
  drugs,
  source = c("All", "WHO", "CHEMBL"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_atc(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
source	Annotation source. One of ‘"All"‘, ‘"WHO"‘, or ‘"CHEMBL"‘. Default is ‘"All"‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_atc()‘ extracts ATC classification information for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a ‘DrugAnnotation‘ object with ATC information stored in ‘object@result‘.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Supported sources are ‘"All"‘, ‘"WHO"‘, and ‘"CHEMBL"‘. Returned annotations may include ATC codes and descriptions across hierarchy levels 1 to 5, depending on the selected source.

Multiple ATC classifications may be associated with a single drug across sources; these are collapsed with ‘|‘ by the shared annotation helpers.

Value

A 'DrugAnnotation' object containing a data frame of ATC classification annotations in 'object@result'.

Examples

```
## Not run:
res_chembl <- get_drug_atc(
  drugs = c("Imatinib", "Gefitinib"),
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_who <- get_drug_atc(
  drugs = drug_df,
  source = "WHO",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_all <- get_drug_atc(
  drugs = c("Imatinib", "Gefitinib"),
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

---

get_drug_bbb *Extract Blood-Brain Barrier Permeability Information*

Description

Retrieves blood-brain barrier permeability annotations for one or more drugs from curated DrugSigNet BBB annotation data.

Usage

```
get_drug_bbb(object = NULL, drugs = NULL, force = FALSE, auth_token = NULL)
## S4 method for signature 'DrugAnnotation'
get_drug_bbb(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when 'drugs' is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a 'Drug' column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a 'Drug' column.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

'get_drug_bbb()' extracts blood-brain barrier permeability classifications for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a 'DrugAnnotation' object with BBB information stored in 'object@result'.

BBB annotations are derived from the Brain-Blood Barrier Database (B3DB), a curated molecular database compiled from experimentally measured blood-brain barrier permeability data reported in published resources. DrugSigNet processes these records into a drug-level annotation resource by collapsing multiple records for the same drug and retaining BBB classifications together with supporting metadata.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using 'force' and 'auth_token'.

The returned annotation may include 'BBB_class' together with available supporting metadata such as compound identifiers, PubChem CID, SMILES, logBB values, decision thresholds, reference identifiers, data group, and comments.

Multiple BBB annotations may be associated with a single drug; these are collapsed with '|' by the shared annotation helpers.

Value

A 'DrugAnnotation' object containing a data frame of BBB permeability annotations in 'object@result'.

References

Meng F, Xi Y, Huang J, Ayers PW. A curated diverse molecular database of blood-brain barrier permeability with chemical descriptors. *Scientific Data*. 2021;8:310. doi:10.1038/s41597021010695

Examples

```
## Not run:
res <- get_drug_bbb(
  drugs = c("Cetirizine", "Pentazocine"),
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)
```

```
drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_df <- get_drug_bbb(
  drugs = drug_df,
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_pipe <- get_drug_bbb(
  object = drug_df,
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

`get_drug_drug_similarity` *Calculate Drug-Drug Structural Similarity*

Description

Computes pairwise structural similarity between drugs using canonical SMILES strings and molecular fingerprints.

Usage

```
get_drug_drug_similarity(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL", "TTD", "B3DB_classification"),
  fingerprint = "standard",
  method = "tanimoto",
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_drug_similarity(object)
```

Arguments

`object`	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`drugs`	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.

source	Annotation source passed to ‘get_drug_smiles()‘. One of ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, ‘"TTD"‘, or ‘"B3DB_classification"‘. Default is ‘"All"‘.
fingerprint	Molecular fingerprint type passed to ‘rcdk‘. Common options include ‘"standard"‘, ‘"extended"‘, and ‘"pubchem"‘. Default is ‘"standard"‘.
method	Similarity metric. Currently only ‘"tanimoto"‘ is supported.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_drug_similarity()‘ first resolves drug names to canonical SMILES strings using ‘get_drug_smiles()‘. It then parses the SMILES strings with ‘rcdk‘, generates molecular fingerprints, and calculates pairwise structural similarity.

The currently supported similarity metric is Tanimoto similarity. Values range from ‘0‘ to ‘1‘, where ‘0‘ indicates no shared fingerprint features and ‘1‘ indicates identical fingerprints.

If a drug has multiple pipe-delimited SMILES annotations, the first non-empty SMILES string is used so that the output keeps one row and one column per input drug. Drugs without valid or parsable SMILES are excluded from the similarity calculation.

Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Value

A ‘DrugAnnotation‘ object. The ‘result‘ slot contains the pairwise structural similarity matrix as a data frame. The input drug-to-SMILES lookup table is stored in ‘object@parameters$drug_smiles_table‘, and the numeric matrix is also stored as ‘attr(object@result, "similarity_matrix")‘.

Examples

```
## Not run:
res <- get_drug_drug_similarity(
  drugs = c("Imatinib", "Gefitinib", "Erlotinib"),
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  Drug = c("Midostaurin", "Pemigatinib")
)

res_ttd <- get_drug_drug_similarity(
  drugs = drug_df,
  source = "TTD",
  fingerprint = "standard",
  method = "tanimoto",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)
```

```
# Pipe-friendly use
res_pipe <- get_drug_drug_similarity(
  object = drug_df,
  source = "OpenTargets",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

`get_drug_indications` *Extract Drug Indications*

Description

Retrieves therapeutic indication annotations for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_indications(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_indications(object)
```

Arguments

`object`	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`drugs`	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`source`	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, or ‘"TTD"‘. Default is ‘"All"‘.
`force`	Logical; force refresh of cached annotation resources when downloading data from Synapse.
`auth_token`	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_indications()‘ extracts disease or therapeutic indication annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a ‘DrugAnnotation‘ object with indication information stored in ‘object@result‘.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Supported sources are ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, and ‘"TTD"‘. When ‘source = "All"‘, indication annotations are combined across available sources.

Multiple indications may be associated with a single drug across sources; these are collapsed with ‘|‘ by the shared annotation helpers.

Value

A ‘DrugAnnotation‘ object containing a data frame of drug indication annotations in ‘object@result‘.

Examples

```
## Not run:
res <- get_drug_indications(
  drugs = c("Imatinib", "Gefitinib"),
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_df <- get_drug_indications(
  drugs = drug_df,
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_ttd <- get_drug_indications(
  drugs = c("Midostaurin", "Pemigatinib"),
  source = "TTD",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

get_drug_moa *Extract Drug Mechanisms of Action*

Description

Retrieves mechanism of action annotations for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_moa(
  object = NULL,
  drugs,
  source = c("All", "OpenTargets", "CHEMBL"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_moa(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
source	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, or ‘"CHEMBL"‘. Default is ‘"All"‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_moa()‘ extracts mechanism of action annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a ‘DrugAnnotation‘ object with MoA information stored in ‘object@result‘.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Supported sources are ‘"All"‘, ‘"OpenTargets"‘, and ‘"CHEMBL"‘. When ‘source = "All"‘, mechanism of action annotations are combined across available sources.

Multiple mechanisms of action may be associated with a single drug across sources; these are collapsed with ‘|‘ by the shared annotation helpers.

Value

A 'DrugAnnotation' object containing a data frame of mechanism of action annotations in 'object@result'.

Examples

```
## Not run:
res <- get_drug_moa(
  drugs = c("Imatinib", "Gefitinib"),
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_df <- get_drug_moa(
  drugs = drug_df,
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

---

get_drug_signature *Retrieve Drug-Induced Gene Expression Signatures*

Description

Retrieves drug-induced gene expression signatures from CMAP or LINCS2 reference databases.

Usage

```
get_drug_signature(
  drug,
  cell = NULL,
  profile_ids = NULL,
  refdb = c("cmap", "lincs2"),
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  auth_token = NULL,
  validate_signature_refdb = TRUE,
  gene_map = NULL,
  n_up = NULL,
  n_down = NULL
)
```

Arguments

drug	Character string specifying the drug name.
cell	Optional character vector specifying one or more cell lines to retain. If ‘NULL‘, all matching cell lines are used.
profile_ids	Optional character vector of exact reference-database HDF5 column names to retain. When supplied, this takes precedence over automatic drug matching and ‘cell‘ filtering.
refdb	Reference database. One of ‘"cmap"‘ or ‘"lincs2"‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.
auth_token	Optional Synapse authentication token used when ‘signature_refdb_mode‘ requests a frozen database. If ‘NULL‘, ‘SYNAPSE_AUTH_TOKEN‘ is used.
validate_signature_refdb	Logical; whether to validate frozen HDF5 reference databases before use. Default is ‘TRUE‘.
gene_map	Optional data frame mapping gene identifiers. Supported columns include Entrez, Ensembl, and gene symbol columns such as ‘Entrez‘, ‘ENTREZID‘, ‘Ensembl‘, ‘ENSEMBL‘, ‘Gene_symbol‘, ‘Symbol‘, or ‘SYMBOL‘.
n_up	Optional positive integer specifying the number of top up-regulated genes to return. If ‘NULL‘, no separate up-regulated gene table is returned.
n_down	Optional positive integer specifying the number of top down-regulated genes to return. If ‘NULL‘, no separate down-regulated gene table is returned.

Details

‘get_drug_signature()‘ extracts perturbation profiles for a specified drug from CMAP or LINCS2 reference databases. By default, all matching treatment profiles are used.

Profile selection can be restricted with ‘cell‘ or ‘profile_ids‘. For CMAP, ‘cell‘ may contain one or more cell lines; the requested cell order is preserved so the resulting profile collapse matches cell-restricted legacy analyses. For LINCS2, ‘profile_ids‘ can be used to select exact HDF5 profile columns such as ‘pert_id__cell__type‘. When ‘profile_ids‘ is supplied, it takes precedence over automatic drug and cell matching.

By default, reference data are loaded from ExperimentHub. Frozen Synapse reference databases can be used by setting ‘signature_refdb_mode = "frozen"‘ or ‘"frozen_force"‘. In this case, the corresponding HDF5 reference database is downloaded through ‘load_signature_refdb()‘ and cached locally.

For LINCS2, drug names are resolved to perturbation identifiers using the ‘lincs_pert_info2‘ metadata table from the ‘signatureSearch‘ package.

Gene identifiers are returned as Entrez IDs and, when available, mapped to Ensembl IDs and HGNC gene symbols. A user-supplied ‘gene_map‘ is used first; missing gene symbols are then resolved through the package fallback mapping.

If ‘n_up‘ or ‘n_down‘ is supplied, treatment profiles are collapsed per gene by selecting the signed value with the largest absolute magnitude, and the top up- or down-regulated genes are returned separately.

Value

A list containing the drug name, reference database, selected treatment profiles, extracted signature table, and optionally ‘up_genes‘ and ‘down_genes‘.

Examples

```
## Not run:
sig <- get_drug_signature(
  drug = "miglitol",
  refdb = "lincs2"
)

sig_cmap <- get_drug_signature(
  drug = "bepridil",
  cell = c("MCF7", "HL60", "PC3"),
  refdb = "cmap",
  n_up = 100,
  n_down = 100
)

sig_frozen <- get_drug_signature(
  drug = "miglitol",
  refdb = "lincs2",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

---

get_drug_smiles *Extract Drug SMILES Strings*

Description

Retrieves canonical SMILES annotations for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_smiles(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL", "TTD", "B3DB_classification"),
```

```
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_smiles(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
source	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, ‘"TTD"‘, or ‘"B3DB_classification"‘. Default is ‘"All"‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_smiles()‘ extracts SMILES annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a ‘DrugAnnotation‘ object with SMILES information stored in ‘object@result‘.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Supported sources are ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, ‘"TTD"‘, and ‘"B3DB_classification"‘. When ‘source = "All"‘, SMILES annotations are combined across available sources.

Multiple SMILES values may be associated with a single drug across sources; these are collapsed with ‘|‘ by the shared annotation helpers.

Value

A ‘DrugAnnotation‘ object containing a data frame of drug SMILES annotations in ‘object@result‘.

Examples

```
## Not run:
res <- get_drug_smiles(
  drugs = c("Imatinib", "Gefitinib"),
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_chembl <- get_drug_smiles(
  drugs = c("Imatinib", "Gefitinib"),
  source = "CHEMBL",
```

```
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_df <- get_drug_smiles(
  drugs = drug_df,
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

`get_drug_status` *Retrieve Drug Regulatory or Development Status*

Description

Retrieves regulatory approval or development status information for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_status(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_status(object)
```

Arguments

`object`	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`drugs`	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
`source`	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, or ‘"TTD"‘. Default is ‘"All"‘.
`force`	Logical; force refresh of cached annotation resources when downloading data from Synapse.
`auth_token`	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

'get_drug_status()' extracts drug status annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a 'DrugAnnotation' object with status information stored in 'object@result'.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Annotation resources are loaded from the local cache or Synapse using 'force' and 'auth_token'.

Supported sources are '"All"', '"OpenTargets"', '"CHEMBL"', and '"TTD"'. Depending on the selected source, 'Highest_status' may represent the highest development phase, approval status, or regulatory status available for each drug.

Value

A 'DrugAnnotation' object containing a data frame of drug status annotations in 'object@result'.

Examples

```
## Not run:
res <- get_drug_status(
  drugs = c("Imatinib", "Gefitinib"),
  source = "OpenTargets",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_df <- get_drug_status(
  drugs = drug_df,
  source = "All",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_ttd <- get_drug_status(
  drugs = c("Midostaurin", "Pemigatinib"),
  source = "TTD",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

`get_drug_synonyms` *Extract Drug Synonyms*

Description

Retrieves alternative names and synonyms for one or more drugs from supported DrugSigNet annotation sources.

Usage

```
get_drug_synonyms(
  object = NULL,
  drugs = NULL,
  source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_synonyms(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
source	Annotation source. One of ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, or ‘"TTD"‘. Default is ‘"All"‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_synonyms()‘ extracts synonym annotations for the supplied drugs. The function accepts a character vector or a one-column data frame and returns a ‘DrugAnnotation‘ object with synonym information stored in ‘object@result‘.

Drug matching is performed by drug name and is case-insensitive through the shared annotation-matching helpers. Depending on the selected source, returned synonyms may include generic names, brand names, chemical names, research names, or other alternative identifiers.

Supported sources are ‘"All"‘, ‘"OpenTargets"‘, ‘"CHEMBL"‘, and ‘"TTD"‘. When ‘source = "All"‘, synonym annotations are combined across available sources. Annotation resources are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Value

A ‘DrugAnnotation‘ object containing a data frame of drug synonym annotations in ‘object@result‘.

Examples

```
## Not run:
res <- get_drug_synonyms(
  drugs = c("Imatinib", "Gefitinib"),
  source = "CHEMBL",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_opentargets <- get_drug_synonyms(
  drugs = drug_df,
  source = "OpenTargets",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Pipe-friendly use
res_pipe <- get_drug_synonyms(
  object = drug_df,
  source = "TTD",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

---

get_drug_trials *Retrieve Clinical Trials for Drugs*

Description

Retrieves clinical trial records involving one or more specified drugs, with optional filtering by disease or condition.

Usage

```
get_drug_trials(
  object = NULL,
  drugs = NULL,
  condition = NULL,
  ignore_case = TRUE,
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'DrugAnnotation'
get_drug_trials(object)
```

Arguments

object	Optional input for pipe-friendly workflows. Used only when ‘drugs‘ is not supplied. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
drugs	Drugs to annotate. Accepts a character vector, a one-column data frame, or a data frame containing a ‘Drug‘ column.
condition	Optional disease or indication used to retrieve matching clinical trial information.
ignore_case	Logical; whether drug and condition matching should be case-insensitive. Default is ‘TRUE‘.
force	Logical; force refresh of cached annotation resources when downloading data from Synapse.
auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

‘get_drug_trials()‘ searches the DrugSigNet clinical trial annotation table for records whose intervention field contains the supplied drug names. Clinical trial data are loaded from the local cache or Synapse using ‘force‘ and ‘auth_token‘.

Drug matching is performed with fixed-string matching against the ‘Interventions‘ field. When ‘ignore_case = TRUE‘, both input drug names and intervention text are converted to lowercase before matching. Leading ‘"Drug:"‘ prefixes in intervention strings are removed during normalization.

If ‘condition‘ is provided, records are first filtered by the ‘Conditions‘ field before drug matching. A ‘matched_drug‘ column is added to indicate which input drug or drugs matched each clinical trial record. Multiple matched drugs are separated with semicolons.

Value

A ‘DrugAnnotation‘ object containing matching clinical trial records in ‘object@result‘.

See Also

‘annotate_drugs()‘

Examples

```
## Not run:
res <- get_drug_trials(
  drugs = c("Imatinib", "Gefitinib"),
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

drug_df <- data.frame(
  drug = c("Cetirizine", "Pentazocine")
)

res_pain <- get_drug_trials(
```

```
  drugs = drug_df,
  condition = "pain",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Pipe-friendly use
res_pipe <- get_drug_trials(
  object = drug_df,
  condition = "cancer",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

`Harmonic_centrality` *Harmonic Centrality for Network-Based Drug Searching*

Description

Prioritizes candidate drugs or target nodes using harmonic centrality on an integrated protein-protein interaction and drug-target network.

Usage

```
Harmonic_centrality(
  object = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  hub_penalty = 0.01,
  max_deg = NULL,
  result_size = NULL,
  target = "drug",
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  filter_paths = TRUE,
  force = FALSE,
  auth_token = NULL
)

## S4 method for signature 'NetworkBased'
Harmonic_centrality(object)
```

Arguments

`object`	Optional ‘NetworkBased‘ object. If ‘NULL‘, a new object is created from the supplied network inputs and method parameters.

`ppi_network` Protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default ‘"gene_gene"‘ network is loaded.

`drug_target_network` Drug-target interaction network. Expected to contain columns ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default ‘"drug_target"‘ network is loaded.

`disease_genes` Data frame containing disease seed genes in a column named ‘gene‘.

`hub_penalty` Numeric hub penalty used to reduce the influence of highly connected nodes. Default is ‘0.01‘.

`max_deg` Maximum allowed node degree. Nodes with degree greater than this value are excluded. If ‘NULL‘, defaults to ‘.Machine$integer.max‘.

`result_size` Number of top-ranked results to return. If ‘NULL‘, it is inferred from the number of unique drugs in ‘drug_target_network‘.

`target` Node type to prioritize. Default is ‘"drug"‘.

`include_indirect_drugs` Logical; whether to include drugs indirectly connected to disease genes. Default is ‘TRUE‘.

`include_non_approved_drugs` Logical; whether to include non-approved or investigational drugs. Default is ‘TRUE‘.

`filter_paths` Logical; whether to restrict graph paths according to the backend implementation. Default is ‘TRUE‘.

`force` Logical; force redownload of default networks from Synapse instead of using the local cache.

`auth_token` Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.

Details

Harmonic centrality scores nodes by summing inverse shortest-path distances to reachable nodes. Unlike classical closeness centrality, it remains defined in disconnected graphs, making it suitable for fragmented biological networks.

The integrated network is built from a protein-protein interaction network (‘ppi_network‘) and a drug-target interaction network (‘drug_target_network‘). Disease genes are used as seed nodes to identify drugs or targets close to the disease module. If either network is ‘NULL‘, the corresponding high-confidence default DrugSigNet network is loaded with ‘load_drugsignet_network()‘ from the local cache or Synapse.

This implementation follows the network-based drug search setting used in CADDIE, where disease seed genes are analyzed on a combined interactome of protein-protein and drug-target interactions.

Value

A ‘NetworkBased‘ object containing harmonic centrality results in ‘object@result‘ and method parameters in ‘object@parameters‘.

Functions

- `Harmonic_centrality(NetworkBased)`: Implements the Harmonic Centrality calculation for the NetworkCentrality object.

References

Hartung M, Anastasi E, Mamdouh ZM, Nogales C, Schmidt HHHW, Baumbach J, Zolotareva O, List M. Cancer driver drug interaction explorer. *Nucleic Acids Research*. 2022;50(W1):W138-W144. doi:10.1093/nar/gkac384

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

res <- Harmonic_centrality(
  disease_genes = disease_genes,
  hub_penalty = 0.01,
  target = "drug"
)

## End(Not run)
```

---

harmonize_signature_results

Harmonize multiple signature drug-search results

Description

Aggregates processed signature drug-search rankings from two or more signature pipeline results. The function is generic: the supplied signature results may have any names and there may be any number of them. For each requested signature family, the original aggregation across all requested reference databases and results is retained. In addition, each complete method/reference combination is aggregated across results. For example, ‘CMAP_CMAP‘ from result 1 is aggregated with ‘CMAP_CMAP‘ from result 2, and the returned ‘PairwiseAcrossResults$CMAP_CMAP‘ entry contains its ‘CRank‘, ‘Dowdall‘, and ‘RRA‘ results. These additional entries are kept in a separate list and do not participate in or change the existing ‘Pairwise‘ or second-stage ‘Harmonized‘ results.

Usage

```
harmonize_signature_results(
  signature_results,
  datasets = c("CMAP", "LINCS", "gCMAP", "Correlation"),
  reference_suffixes = c("CMAP", "LINCS2"),
  methods = c("CRank", "Dowdall", "RRA"),
```

```
  prior = 0.093,
  num_bin = 200,
  ties_method = "max"
)
```

Arguments

signature_results	Named list of signature pipeline results. Each entry may be a DrugSearchingPipeline object or a list containing DrugSearching$Processed.
datasets	Character vector of signature method families to harmonize.
reference_suffixes	Character vector of processed-result suffixes to harmonize separately for every dataset.
methods	Character vector of rank aggregation methods. Supported values are "CRank", "Dowdall", and "RRA".
prior	Prior passed to CRank().
num_bin	Number of bins passed to CRank().
ties_method	Ties method passed to rank aggregation methods.

Details

The first-stage aggregation deliberately reproduces the behaviour of the original Harmonize_Signature_Network() implementation. In particular, it does not collapse duplicate perturbations and does not replace missing values with zero before CRank/Dowdall/RRA.

A second-stage Harmonized aggregation is produced only when at least two signature families are available. With one family there is no second independent ranking to aggregate, so Harmonized is an empty list.

Value

A list with Pairwise, PairwiseAcrossResults, and Harmonized entries. Pairwise retains the original family-level results. PairwiseAcrossResults is keyed by complete processed-method names such as CMAP_CMAP; each entry contains the requested CRank, Dowdall, and RRA objects. RRA result tables consistently use Drug as the identifier column.

---

integrateSignatureNetwork

Integrate Signature- and Network-Based Drug Rankings

Description

Integrates current 'DrugSearchingPipeline' outputs from signature-based and network-based workflows into a single 'DrugSearchingPipeline' object with consensus CRank, Dowdall, and Robust Rank Aggregation (RRA) rankings.

The function:

- Accepts either 'DrugSearchingPipeline' S4 objects or legacy list results
- Expands pipe-delimited network drug entries before matching overlaps
- Detects overlapping drugs between both pipelines
- Automatically harmonizes common drug identifier columns
- Combines pairwise signature ranks with raw centrality-derived, stretched network ranks
- Includes raw Network_proximity and Diffusion ranks when present
- Performs CRank, Dowdall, and RRA aggregation
- Builds updated 'RankAggregation', 'DrugAnnotation', and 'Visualization' slots

Usage

```
integrateSignatureNetwork(
  signature_res,
  network_res,
  ties_method = "max",
  prior = 0.093,
  num_bin = 200,
  top_k = 100,
  trial_condition = NULL,
  target_id_from = NULL,
  drug_annotation_source = c("All", "OpenTargets", "CHEMBL", "TTD"),
  force = FALSE,
  auth_token = NULL,
  run_drug_annotation = TRUE,
  run_visualization = TRUE
)
```

Arguments

signature_res	A 'DrugSearchingPipeline' object returned by `drugSignaturePipeline()` or a legacy result list containing signature rank aggregation results. The direct output from `harmonize_signature_results()` is also supported. Integration uses 'RankAggregation$Pairwise' signature ranks for integrated aggregation and 'RankAggregation$Harmonized' ranks for the signature columns when present.
network_res	A 'DrugSearchingPipeline' object returned by `drugNetworkPipeline()` or a legacy result list containing 'RankAggregation$Network_Harmonized'.
ties_method	Character tie-handling method ("max", "min", "dense", etc.).
prior	Numeric prior used in CRank. Default is 0.093.
num_bin	Integer number of bins for CRank discretization. Default is 200.

top_k Integer number of top drugs used for enrichment.

trial_condition
Optional condition filter passed to annotate_drugs().

target_id_from Optional AnnotationDbi keytype or supported alias describing target identifiers used for functional enrichment. Default is NULL, which treats targets as HGNC symbols unless Ensembl gene IDs are detected.

drug_annotation_source
Source used in annotate_drugs(). One of "All", "OpenTargets", "CHEMBL", or "TTD". Default is "All".

force Logical; passed to annotate_drugs() and annotation layer helpers to force Synapse cache refresh where supported.

auth_token Optional Synapse authentication token passed to annotate_drugs() and annotation layer helpers.

run_drug_annotation
Logical; if TRUE, annotate integrated drugs and run target-set enrichment. If FALSE, the DrugAnnotation section is omitted ('NULL'). Default is TRUE.

run_visualization
Logical; if TRUE, build visualization metadata and plots. If FALSE, the Visualization section is omitted ('NULL'). Default is TRUE.

Value

A 'DrugSearchingPipeline' S4 object with 'type = "integration"'. The 'RankAggregation' slot contains 'Signature_Network_CRank', 'Signature_Network_Dowdall', 'Signature_Network_RRA', and 'Signature_Network_Harmonized'. The integration result keeps 'DrugSearching' empty and returns only rank aggregation, annotation, and visualization outputs.

---

lincs_expr_inst_info *LINCS Expression-Instance Metadata*

Description

Metadata for expression instances in the bundled LINCS reference data.

Format

A data frame with one row per expression instance and columns for signature, perturbagen, cell, dose, and treatment metadata where available.

lincs_method *LINCS Method for Signature-Based Drug Searching*

Description

Performs gene expression signature-based drug repurposing using the LINCS algorithm.

Usage

```
lincs_method(
  object = NULL,
  upset,
  downset,
  ref_db,
  tau = FALSE,
  sortby = "WTCS",
  chunk_size = 5000,
  GeneType = "reference",
  signature_refdb_mode = c("default", "frozen", "frozen_force"),
  auth_token = NULL,
  validate_signature_refdb = TRUE
)

## S4 method for signature 'SignatureBased'
lincs_method(object)
```

Arguments

object	Optional ‘SignatureBased‘ object. If ‘NULL‘, a new object is created from ‘upset‘, ‘downset‘, and ‘ref_db‘.
upset	Character vector of up-regulated Entrez gene IDs. May be ‘NULL‘ if ‘downset‘ is provided.
downset	Character vector of down-regulated Entrez gene IDs. May be ‘NULL‘ if ‘upset‘ is provided.
ref_db	Reference database. Use ‘"cmap"‘ or ‘"lincs2"‘, or provide a local HDF5 path returned by ‘load_signature_refdb()‘ to use a frozen Synapse reference database.
tau	Logical; whether to apply tau ranking. Default is ‘FALSE‘.
sortby	Character string specifying the score used to sort LINCS results. Default is ‘"WTCS"‘.
chunk_size	Integer; chunk size used by the LINCS search. Default is ‘5000‘.
GeneType	Character string specifying the gene type used by the reference database search. Default is ‘"reference"‘.
signature_refdb_mode	Signature reference database mode. Use ‘"default"‘ for the default ExperimentHub reference, ‘"frozen"‘ for the cached Synapse HDF5 reference database, or ‘"frozen_force"‘ to redownload the frozen database.

auth_token Optional Synapse authentication token used when ‘signature_refdb_mode‘ requests a frozen database. If ‘NULL‘, ‘SYNAPSE_AUTH_TOKEN‘ is used.

validate_signature_refdb
Logical; whether to validate frozen HDF5 reference databases before use. Default is ‘TRUE‘.

Details

The query may contain up-regulated genes (‘upset‘), down-regulated genes (‘downset‘), or both. At least one of ‘upset‘ or ‘downset‘ must be provided; they cannot both be ‘NULL‘.

‘ref_db‘ can be ‘"cmap"‘ or ‘"lincs2"‘. A local HDF5 path returned by ‘load_signature_refdb()‘ can also be supplied to use a frozen Synapse reference database. Frozen databases can also be fetched directly by setting ‘signature_refdb_mode = "frozen"‘ or ‘"frozen_force"‘ and providing ‘auth_token‘.

Value

A ‘SignatureBased‘ object containing the LINCS search results and query metadata.

Functions

- `lincs_method(SignatureBased)`: Implements the LINCS method for Signature-Based Drug Searching.

Examples

```
## Not run:
upset <- c("7157", "1956", "5290")
downset <- c("7422", "4318", "348")

# Search using the default LINCS reference database
res <- lincs_method(
  upset = upset,
  downset = downset,
  ref_db = "lincs2"
)

# Run a one-sided query
res_up <- lincs_method(
  upset = upset,
  downset = NULL,
  ref_db = "lincs2"
)

# Automatically fetch and use a frozen Synapse reference database
res_frozen <- lincs_method(
  upset = upset,
  downset = downset,
  ref_db = "lincs2",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
```

```
)

# Reuse a local frozen HDF5 reference database
lincs_ref <- load_signature_refdb(
  ref_db = "lincs2",
  signature_refdb_mode = "frozen",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res_local <- lincs_method(
  upset = upset,
  downset = downset,
  ref_db = lincs_ref
)

## End(Not run)
```

`lincs_pert_info` *LINCS Perturbagen Metadata*

Description

Perturbagen annotations for the bundled LINCS reference data.

Format

A data frame with one row per perturbagen and identifier and annotation columns.

`lincs_pert_info2` *Curated LINCS Perturbagen Metadata*

Description

A curated subset of LINCS perturbagen annotations used to map perturbagen identifiers to names.

Format

A data frame with one row per perturbagen and columns including perturbagen identifiers and names.

lincs_rank_score *Rank LINCS Signature Results*

Description

Converts raw results from [lincs_method()] into the standardized ranking format used by the signature pipeline.

Usage

```
lincs_rank_score(lincs_signature, ties_method = "max")
```

Arguments

lincs_signature	A data frame of raw LINCS signature results.
ties_method	Character string passed to [base::rank()], or ‘"dense"‘.

Value

A data frame of standardized LINCS rank scores.

lincs_sig_info *LINCS Signature Metadata*

Description

Signature-level annotations for the bundled LINCS reference data.

Format

A data frame with one row per signature and columns describing its perturbagen and experimental context.

listOrMat-class *List-or-matrix class union*

Description

An internal S4 class union allowing a slot to contain either a list or a matrix.

load_drugsignet_network *Download and Cache DrugSigNet Network Data*

Description

Downloads DrugSigNet network resources from Synapse and caches them in the user cache directory.

Usage

```
load_drugsignet_network(
  network = c("drug_target", "gene_gene", "all"),
  force = FALSE,
  auth_token = NULL
)
```

Arguments

network	Network resource to load. One of '"drug_target"', '"gene_gene"', or '"all"'.
force	Logical; if 'TRUE', redownload the selected network even when a cached version is available or current. Default is 'FALSE'.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

'load_drugsignet_network()' retrieves DrugSigNet network resources used by network-based drug repurposing methods, including the drug-target interaction network and the integrated gene-gene interaction network.

Cached files are stored under 'tools::R_user_dir("DrugSigNet", "cache")'. When a Synapse authentication token is available, the function checks the remote Synapse file version and updates the local cache when needed. If no token is available but a cached file already exists, the cached file is used without a remote version check.

Use 'force = TRUE' to redownload the selected network from Synapse. This requires a valid Synapse authentication token.

Value

A data frame when a single network is requested, or a named list of data frames when 'network = "all"'.

Examples

```
## Not run:
# Load the default drug-target network
drug_target <- load_drugsignet_network(
  network = "drug_target",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Load the default gene-gene interaction network
gene_gene <- load_drugsignet_network(
  network = "gene_gene",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Load all available network resources
networks <- load_drugsignet_network(
  network = "all",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

# Use an existing local cache without providing a token
cached_drug_target <- load_drugsignet_network("drug_target")

## End(Not run)
```

---

load_signature_refdb *Download and Cache Frozen Signature Reference Databases*

Description

Downloads frozen DrugSigNet signature reference databases from Synapse and caches them in the user cache directory.

Usage

```
load_signature_refdb(
  refdb = c("cmap", "lincs2"),
  force = FALSE,
  auth_token = NULL,
  validate_hdf5 = TRUE
)
```

Arguments

refdb	Reference database to download. One of ‘"cmap"‘ or ‘"lincs2"‘.
force	Logical; if ‘TRUE‘, redownload the file even when the cached Synapse version is current. Default is ‘FALSE‘.

auth_token	Optional Synapse authentication token. If ‘NULL‘, the ‘SYNAPSE_AUTH_TOKEN‘ environment variable is used.
validate_hdf5	Logical; whether to validate the cached or downloaded file with ‘rhdf5::h5ls()‘ before returning it. Default is ‘TRUE‘.

Details

‘load_signature_refdb()‘ retrieves a fixed HDF5 reference database for signature-based drug searching. The returned file path can be supplied to DrugSigNet signature methods, such as ‘cmap_method()‘, ‘lincs_method()‘, ‘gcmap_method()‘, and ‘correlation_method()‘, when a reproducible frozen reference database is preferred over package defaults.

Cached files are stored under ‘tools::R_user_dir("DrugSigNet", "cache")‘. On each call, the function compares the cached metadata with the current Synapse file version. If the cached version is current, the local file is reused. If the remote version has changed, or if ‘force = TRUE‘, the file is downloaded again and the cache metadata is updated.

When ‘validate_hdf5 = TRUE‘, the downloaded or cached file is checked with ‘rhdf5::h5ls()‘ before the path is returned.

Value

Character path to the cached HDF5 reference database.

Examples

```
## Not run:
cmap_ref <- load_signature_refdb(
  refdb = "cmap",
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

res <- cmap_method(
  upset = c("7157", "1956", "5290"),
  downset = c("7422", "4318", "348"),
  ref_db = cmap_ref
)

lincs_ref <- load_signature_refdb(
  refdb = "lincs2",
  force = TRUE,
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

## End(Not run)
```

load_synapse_annotations *Load DrugSigNet Drug Annotation Data*

Description

Downloads DrugSigNet drug annotation data from Synapse and caches it in the user cache directory.

Usage

```
load_synapse_annotations(force = FALSE, auth_token = NULL)
```

Arguments

force	Logical; if 'TRUE', redownload the Synapse annotation file even when a current local cache is available. Default is 'FALSE'.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

'load_synapse_annotations()' retrieves the DrugSigNet drug annotation reference object used by annotation helper functions. The object contains annotation tables for indications, mechanisms of action, ATC codes, clinical trials, adverse events, synonyms, SMILES strings, approval status, and blood-brain barrier classifications.

Cached files are stored under 'tools::R_user_dir("DrugSigNet", "cache")'. On each call, the function compares the cached metadata with the current Synapse file version. If the cached version is current, the local file is reused. If the remote version has changed, or if 'force = TRUE', the file is downloaded again and the cache metadata is updated.

This function requires the optional 'synapser' package and access to the DrugSigNet Synapse annotation file.

Value

The DrugSigNet drug annotation object loaded from the local cache or downloaded from Synapse.

Examples

```
## Not run:
annotation <- load_synapse_annotations(
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)

annotation <- load_synapse_annotations(
  force = TRUE,
  auth_token = Sys.getenv("SYNAPSE_AUTH_TOKEN")
)
```

```
## End(Not run)
```

NetworkBased-class *Create a SignatureBasedDrugSearching Object*

Description

A helper function to create an instance of the ‘SignatureBasedDrugSearching‘ class.

Arguments

signature_result	A ‘data.frame‘ containing the results of the search.
query	A ‘list‘ containing the up-regulated (‘upset‘) and down-regulated (‘downset‘) gene sets.
signature_method	A ‘character‘ string representing the method used (e.g., "CMAP").
refdb	A ‘character‘ string specifying the reference database used (e.g., "cmap").

Value

A ‘SignatureBasedDrugSearching‘ object.

Network_proximity *Network Proximity for Network-Based Drug Searching*

Description

Prioritizes candidate drugs using network proximity between drug targets and disease genes in an integrated interactome.

Usage

```
Network_proximity(
  object = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  n_simulations = 1000L,
  n_workers = 1L,
  result_size = NULL,
```

```
  random_seed = 42L,
  force = FALSE,
  auth_token = NULL
)
```

Arguments

object	Optional 'NetworkBased' object. If 'NULL', a new object is created from the supplied network inputs and method parameters.
ppi_network	Protein-protein interaction network. Expected to contain columns 'gene1' and 'gene2'. If 'NULL', the default '"gene_gene"' network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns 'ID', 'Drug', 'Target', and 'Group'. If 'NULL', the default '"drug_target"' network is loaded.
disease_genes	Data frame containing disease seed genes in a column named 'gene'.
include_indirect_drugs	Logical; whether to include drugs indirectly connected to disease genes. Default is 'TRUE'.
include_non_approved_drugs	Logical; whether to include non-approved or investigational drugs. Default is 'TRUE'.
n_simulations	Number of random simulations used to estimate the proximity background distribution. Default is '1000'.
n_workers	Number of CPU workers used for the simulation procedure. Default is '1'.
result_size	Number of top-ranked results to return. If 'NULL', it is inferred from the number of unique drugs in 'drug_target_network'.
random_seed	Integer random seed used for reproducible simulations. Default is '42'.
force	Logical; force redownload of default networks from Synapse instead of using the local cache.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

Network proximity measures how close drug targets are to disease-associated genes in the interactome. Drugs whose targets are closer to the disease module are prioritized as candidates for repurposing or follow-up analysis.

The integrated network is built from a protein-protein interaction network ('ppi_network') and a drug-target interaction network ('drug_target_network'). Disease genes define the disease module. If either network is 'NULL', the corresponding high-confidence default DrugSigNet network is loaded with 'load_drugsignet_network()' from the local cache or Synapse.

The method estimates a background distribution using random simulations and ranks drugs by their proximity to the supplied disease genes.

This implementation follows the network medicine framework used to identify drug-repurposing opportunities for COVID-19.

Value

A 'NetworkBased' object containing network proximity results in 'object@result' and method parameters in 'object@parameters'.

References

Morselli Gysi D, do Valle Í, Zitnik M, Ameli A, Gan X, Varol O, Ghiassian SD, Patten JJ, Davey RA, Loscalzo J, Barabási AL. Network medicine framework for identifying drug-repurposing opportunities for COVID-19. *Proceedings of the National Academy of Sciences of the United States of America*. 2021;118(19):e2025581118. doi:10.1073/pnas.2025581118

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

# Run with default DrugSigNet networks
res <- Network_proximity(
  disease_genes = disease_genes,
  n_simulations = 1000,
  n_workers = 1,
  random_seed = 42
)

# Run with user-provided networks
ppi_network <- data.frame(
  gene1 = c("ENSG00000130203", "ENSG00000142192"),
  gene2 = c("ENSG00000171867", "ENSG00000198786")
)

drug_target_network <- data.frame(
  ID = c("DB0001", "DB0002"),
  Drug = c("drug_a", "drug_b"),
  Target = c("ENSG00000130203", "ENSG00000171867"),
  Group = c("approved", "approved")
)

res_custom <- Network_proximity(
  ppi_network = ppi_network,
  drug_target_network = drug_target_network,
  disease_genes = disease_genes,
  n_simulations = 1000,
  n_workers = 1,
  random_seed = 42
)

## End(Not run)
```

Network_proximity,NetworkBased-method *Network Proximity Method for Network-Based Drug Search*

Description

Method for the Network Proximity drug search using a network-based proximity algorithm. The method performs simulations and ranks drugs based on their proximity to disease genes.

Usage

```
## S4 method for signature 'NetworkBased'
Network_proximity(object)
```

Arguments

object	A NetworkBased object containing the networks, disease genes, and network-proximity parameters.

Value

The same NetworkBased object with the results of the Network Proximity analysis.

optimize_parameters *Optimize Parameters for DrugSigNet Ranking Methods*

Description

Performs grid-based parameter optimization for selected DrugSigNet ranking and rank aggregation methods.

Usage

```
optimize_parameters(
  method = c("TrustRank", "Harmonic_centrality", "CRank"),
  input_data = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes = NULL,
  hub_penalty = seq(0, 1, 0.1),
  damping_factor = seq(0, 1, 0.1),
  prior = seq(0.001, 0.1, 0.001),
  num_bin = seq(100, 1000, 100),
  num_iter = 1000,
  ties_method = "max",
```

```
  reverse = FALSE,
  cancer_drugs = NULL,
  positive_drugs = NULL,
  negative_drugs = NULL,
  result_size = NULL,
  n_workers = 8,
  force = FALSE,
  auth_token = NULL
)
```

Arguments

method	Ranking method to optimize. One of '"TrustRank"', '"Harmonic_centrality"', or '"CRank"'.
input_data	Data frame used for CRank rank aggregation. Required when 'method = "CRank"'.
ppi_network	Protein-protein interaction network. Expected to contain columns 'gene1' and 'gene2'. If 'NULL', the default '"gene_gene"' network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns 'ID', 'Drug', 'Target', and 'Group'. If 'NULL', the default '"drug_target"' network is loaded.
disease_genes	Data frame containing disease seed genes in a column named 'gene'.
hub_penalty	Numeric vector of hub penalty values to test for network methods. Default is 'seq(0, 1, 0.1)'.
damping_factor	Numeric vector of damping factors to test for 'TrustRank'. Default is 'seq(0, 1, 0.1)'.
prior	Numeric vector of CRank prior values to test. Default is 'seq(0.001, 0.1, 0.001)'.
num_bin	Integer vector of CRank bin counts to test. Default is 'seq(100, 1000, 100)'.
num_iter	Number of CRank iterations. Default is '1000'.
ties_method	Method used to resolve ties during rank conversion. One of '"max"', '"min"', '"average"', '"first"', '"last"', '"random"', or '"dense"'. Default is '"max"'.
reverse	Logical; if 'TRUE', smaller input values are treated as better, which is appropriate when numeric columns already represent ranks. If 'FALSE', larger input values are treated as better. Default is 'FALSE'.
cancer_drugs	Optional vector of disease-relevant drugs reserved for benchmarking workflows.
positive_drugs	Optional vector of positive-control drugs reserved for benchmarking workflows.
negative_drugs	Optional vector of negative-control drugs reserved for benchmarking workflows.
result_size	Number of top-ranked results to return. If 'NULL', it is inferred from the number of unique drugs in 'drug_target_network'.
n_workers	Number of parallel workers. Network methods are run sequentially for Python/reticulate backend stability; CRank optimization can use multiple workers. Default is '8'.
force	Logical; force redownload of default networks from Synapse instead of using the local cache.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

'optimize_parameters()' evaluates parameter combinations for one selected method and returns the corresponding ranked results. It supports network methods ('TrustRank' and 'Harmonic_centrality') and the CRank rank aggregation method.

For 'TrustRank', the grid is defined by 'hub_penalty' and 'damping_factor'. For 'Harmonic_centrality', the grid is defined by 'hub_penalty'. For 'CRank', the grid is defined by 'prior' and 'num_bin'.

Network methods use the resolved PPI and drug-target networks and require 'disease_genes'. If either network is 'NULL', default DrugSigNet networks can be loaded from the local cache or Synapse. Because network methods rely on Python/reticulate graph backends, they are run sequentially for stability. CRank optimization can use multiple workers.

Value

A named list of optimization results. Each element contains the parameter label, tested parameter values, and the corresponding method result.

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3)
)

crank_grid <- optimize_parameters(
  method = "CRank",
  input_data = rank_df,
  prior = c(0.01, 0.05, 0.093),
  num_bin = c(100, 200),
  reverse = TRUE,
  n_workers = 2
)

disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192")
)

trust_grid <- optimize_parameters(
  method = "TrustRank",
  disease_genes = disease_genes,
  hub_penalty = c(0, 0.01, 0.1),
  damping_factor = c(0.85, 0.95),
  result_size = 100
)

## End(Not run)
```

plot_all *Plot Multiple DrugSigNet Visualizations*

Description

Runs multiple DrugSigNet plotting helpers in one call.

Usage

```
plot_all(
  plot_inputs,
  file_type = c("pdf", "png", "jpeg", "svg"),
  output_dir = NULL,
  file_prefix = "",
  continue_on_error = TRUE,
  verbose = TRUE
)
```

Arguments

plot_inputs	Named list of plot inputs. Names must be supported plot keys. Each value can be a data frame, matrix, or a named list of arguments for the underlying plotting function.
file_type	Output file format. One of ‘"pdf"‘, ‘"png"‘, ‘"svg"‘, or ‘"jpeg"‘. Default is ‘"pdf"‘.
output_dir	Optional output directory for saved plots. If ‘NULL‘, each plotting function uses its default saving behavior.
file_prefix	Optional prefix added to generated file names when ‘output_dir‘ is provided.
continue_on_error	Logical; if ‘TRUE‘, failed plots are collected and processing continues. If ‘FALSE‘, the first failure stops execution.
verbose	Logical; if ‘TRUE‘, progress messages are printed.

Details

‘plot_all()‘ is a convenience wrapper for generating several DrugSigNet plots from a named list of inputs. Each list element name must match one supported plot key. Each element can either be the main input object for that plot or a named list of arguments passed directly to the corresponding plotting function.

Supported plot keys are ‘drug_status‘, ‘drug_indications‘, ‘drug_similarity‘, ‘drug_hierarchy‘, ‘top_k_hits‘, ‘top_k_overlap‘, ‘enriched_terms‘, ‘rank_agreement‘, and ‘rank_distribution_scatter‘.

If ‘output_dir‘ is provided, file names are generated automatically using ‘file_prefix‘ and the plot key unless a plot-specific ‘file_name‘ is supplied. Failed plots are collected in the returned ‘errors‘ list when ‘continue_on_error = TRUE‘.

Value

A named list with two elements:

plots Successfully generated plot objects or plot results.

errors Named list of error messages for failed plots.

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Status = c("positive", NA, "positive"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3)
)

indication_df <- data.frame(
  Drug = c("drug_a", "drug_b"),
  indication = c("Cancer|Tumor", "Diabetes")
)

res <- plot_all(
  plot_inputs = list(
    top_k_hits = list(
      drug_ranks_df = rank_df,
      plottype = "Barplot"
    ),
    top_k_overlap = rank_df,
    drug_indications = indication_df
  ),
  file_type = "pdf",
  output_dir = "plots",
  file_prefix = "DrugSigNet_"
)

res$plots
res$errors

## End(Not run)
```

---

plot_drug_signature_overlay

Plot Drug Signature Overlay

Description

Visualizes a drug-centered subgraph with drug targets, disease seed genes, and drug-signature genes overlaid.

Usage

```
plot_drug_signature_overlay(
  object = NULL,
  Drug = NULL,
  refdb = NULL,
  res = NULL,
  disease_genes = NULL,
  gene_map = NULL,
  subgraph_result = NULL,
  disease_seed_genes = NULL,
  drug_signature_genes = NULL,
  drug_signature_use = c("top", "up", "down", "signature"),
  symbol_map = NULL,
  max_signature_path_distance = Inf,
  include_all_drug_targets = FALSE,
  node_alpha = 0.85,
  seed = 1,
  layout = "fr",
  file_type = "pdf",
  file_name = NULL,
  width = 10,
  height = 10,
  units = "in"
)

## S4 method for signature 'PlotObject'
plot_drug_signature_overlay(
  object = NULL,
  Drug = NULL,
  refdb = NULL,
  res = NULL,
  disease_genes = NULL,
  gene_map = NULL,
  subgraph_result = NULL,
  disease_seed_genes = NULL,
  drug_signature_genes = NULL,
  drug_signature_use = c("top", "up", "down", "signature"),
  symbol_map = NULL,
  max_signature_path_distance = Inf,
  include_all_drug_targets = FALSE,
  node_alpha = 0.85,
  seed = 1,
  layout = "fr",
  file_type = "pdf",
  file_name = NULL,
  width = 10,
  height = 10,
  units = "in"
```

```
)
```

Arguments

object	Optional 'PlotObject'. For pipe-friendly use, a data frame can also be supplied as 'object' when 'drug_ranks_df' is omitted.
Drug, refdb, res	Optional workflow-style inputs. When 'res' is supplied, 'subgraph_result' is taken from 'res[[Drug]][[refdb]]'.
disease_genes	Optional disease-gene table used as 'disease_seed_genes' when 'res' is supplied.
gene_map	Optional gene ID-to-symbol mapping table used when 'symbol_map' is not supplied.
subgraph_result	Result returned by 'extract_drug_subgraph()'.
disease_seed_genes	Disease seed genes as a character vector or one-column data frame.
drug_signature_genes	Drug-signature genes as a character vector, one-column data frame, or result from 'get_drug_signature()'.
drug_signature_use	Component of a 'get_drug_signature()' result to use. One of '"top"', '"up"', '"down"', or '"signature"'.
symbol_map	Optional two-column data frame with graph gene IDs and gene symbols.
max_signature_path_distance	Maximum shortest-path distance from drug targets to disease or signature genes. Default is 'Inf'.
include_all_drug_targets	Logical; if 'TRUE', include all direct drug targets in the visualization. Default is 'FALSE'.
node_alpha	Node transparency. Default is '0.85'.
seed	Random seed used for graph layout. Default is '1'.
layout	Graph layout passed to 'ggraph::create_layout()'. Default is '"fr"'.
file_type	Output file format. One of '"pdf"', '"png"', '"svg"', or '"jpeg"'. Default is '"pdf"'.
file_name	Optional output file name without extension. If 'NULL', the plot is returned without saving.
width, height	Plot dimensions. Defaults are '20' and '15'.
units	Units for saved plot dimensions. One of '"in"', '"cm"', '"mm"', or '"px"'. Default is '"in"'.

Details

'plot_drug_signature_overlay()' plots an extracted drug-centered PPI subgraph returned by 'extract_drug_subgraph()'. Drug-target edges are retained to show the target context, while disease seed and drug-signature evidence are highlighted on the graph.

Disease seed genes can be supplied as a character vector or one-column data frame. Drug-signature genes can be supplied as a character vector, one-column data frame, or a result from ‘get_drug_signature()‘.

When direct target evidence is unavailable, the function can use one-step disease seed neighbors or shortest paths between drug targets and drug-signature genes.

Value

A list containing the plot, graph, node table, edge table, disease seed evidence, drug-signature evidence, shortest-path information, and evidence mode summaries.

Examples

```
## Not run:
subg <- extract_drug_subgraph(
  disease_genes_df = disease_genes,
  drug = "miglitol",
  k_hops = 2
)

drug_sig <- get_drug_signature(
  drug = "miglitol",
  refdb = "lincs2",
  n_up = 100,
  n_down = 100
)

seed_df <- data.frame(
  gene = c("ENSG00000141510", "ENSG00000171862")
)

overlay <- plot_drug_signature_overlay(
  subgraph_result = subg,
  disease_seed_genes = seed_df,
  drug_signature_genes = drug_sig,
  drug_signature_use = "top"
)

overlay$plot

## End(Not run)
```

---

plot_drug_similarity *Plot Drug-Drug Similarity Matrix*

Description

Visualizes a drug-drug structural similarity matrix as a heatmap or dendrogram.

Usage

```
plot_drug_similarity(
  object = NULL,
  similarity_matrix,
  plot_type = c("heatmap", "dendrogram"),
  heatmap_type = c("triangle", "full"),
  file_type = "pdf",
  file_name = NULL,
  width = 20,
  height = 15,
  label_size = 3,
  units = "in"
)

## S4 method for signature 'PlotObject'
plot_drug_similarity(object, plot_type, heatmap_type, label_size)
```

Arguments

object	Optional ‘PlotObject‘. For pipe-friendly use, a data frame can also be supplied as ‘object‘ when ‘drug_ranks_df‘ is omitted.
similarity_matrix	Square numeric similarity matrix.
plot_type	Plot type. One of ‘"heatmap"‘ or ‘"dendrogram"‘.
heatmap_type	Heatmap layout. One of ‘"triangle"‘ or ‘"full"‘. Default is ‘"triangle"‘.
file_type	Output file format. One of ‘"pdf"‘, ‘"png"‘, ‘"svg"‘, or ‘"jpeg"‘. Default is ‘"pdf"‘.
file_name	Optional output file name without extension. If ‘NULL‘, the plot is returned without saving.
width, height	Plot dimensions. Defaults are ‘20‘ and ‘15‘.
label_size	Text size for heatmap-cell labels. Default is ‘3‘.
units	Units for saved plot dimensions. One of ‘"in"‘, ‘"cm"‘, ‘"mm"‘, or ‘"px"‘. Default is ‘"in"‘.

Details

‘plot_drug_similarity()‘ accepts a ‘DrugAnnotation‘ object, numeric matrix, or data frame. Heatmap mode returns a ‘ggplot‘ object. Dendrogram mode plots hierarchical clustering based on ‘1 - similarity‘ and invisibly returns the ‘hclust‘ object.

Value

A ‘ggplot‘ object for heatmaps, or an invisible ‘hclust‘ object for dendrograms.

Examples

```
## Not run:
sim_mat <- matrix(
  c(1, 0.7, 0.2,
    0.7, 1, 0.3,
    0.2, 0.3, 1),
  nrow = 3,
  dimnames = list(
    c("drug_a", "drug_b", "drug_c"),
    c("drug_a", "drug_b", "drug_c")
  )
)

plot_drug_similarity(sim_mat, plot_type = "heatmap")
plot_drug_similarity(sim_mat, plot_type = "dendrogram")

## End(Not run)
```

---

plot_rank_distribution

Plot Pairwise Rank Distributions

Description

Creates pairwise scatter plots comparing rank values across selected ranking methods.

Usage

```
plot_rank_distribution(
  object = NULL,
  drug_ranks_df = NULL,
  methods = NULL,
  top_k = NULL,
  point_size = 1.5,
  alpha = 0.6,
  file_type = "pdf",
  file_name = NULL,
  width = 12,
  height = 10,
  units = "in"
)

## S4 method for signature 'PlotObject'
plot_rank_distribution(object, methods, top_k, point_size, alpha)
```

Arguments

object	Optional 'PlotObject'. For pipe-friendly use, a data frame can also be supplied as 'object' when 'drug_ranks_df' is omitted.
drug_ranks_df	Data frame containing 'Drug', optional 'Status', optional facet columns, and numeric rank columns. Smaller ranks are better.
methods	Optional character vector of ranking-method columns to plot. If 'NULL', all numeric columns are used.
top_k	Positive rank threshold. Default is '100'.
point_size	Point size. Default is '1.5'.
alpha	Point transparency in '(0, 1]'. Default is '0.6'.
file_type	Output file format. One of '"pdf"', '"png"', '"svg"', or '"jpeg"'. Default is '"pdf"'.
file_name	Optional output file name without extension. If 'NULL', the plot is returned without saving.
width, height	Plot dimensions. Defaults are '20' and '15'.
units	Units for saved plot dimensions. One of '"in"', '"cm"', '"mm"', or '"px"'. Default is '"in"'.

Details

'plot_rank_distribution()' compares numeric ranking columns by plotting all pairwise method combinations. If 'top_k' is supplied, only drugs appearing within the top-k threshold in at least one selected method are retained.

Value

A 'ggplot' object.

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3),
  Method3 = c(3, 1, 2)
)

plot_rank_distribution(rank_df)

plot_rank_distribution(
  drug_ranks_df = rank_df,
  methods = c("Method1", "Method2"),
  top_k = 100
)

## End(Not run)
```

plot_top_k_overlap *Plot Top-K Overlap Across Drug Ranking Methods*

Description

Visualizes pairwise overlap among top-ranked drugs across ranking methods.

Usage

```
plot_top_k_overlap(
  object = NULL,
  drug_ranks_df = NULL,
  top_k = 100,
  file_type = "pdf",
  file_name = NULL,
  status_df = NULL,
  status_col = "Status",
  width = 20,
  height = 15,
  label_size = 3,
  base_size = 11,
  heatmap_type = c("triangle", "full"),
  split_grouped_drugs = FALSE,
  drug_sep = "\\|",
  facet_cols = NULL,
  facet_type = c("wrap", "grid"),
  facet_rows = NULL,
  facet_grid_cols = NULL,
  units = "in"
)

## S4 method for signature 'PlotObject'
plot_top_k_overlap(
  object,
  top_k,
  status_df,
  status_col,
  label_size,
  base_size,
  heatmap_type,
  split_grouped_drugs,
  drug_sep,
  facet_cols,
  facet_type,
  facet_rows,
  facet_grid_cols
)
```

Arguments

object	Optional ‘PlotObject‘. For pipe-friendly use, a data frame can also be supplied as ‘object‘ when ‘drug_ranks_df‘ is omitted.
drug_ranks_df	Data frame containing ‘Drug‘, optional ‘Status‘, optional facet columns, and numeric rank columns. Smaller ranks are better.
top_k	Positive rank threshold. Default is ‘100‘.
file_type	Output file format. One of ‘"pdf"‘, ‘"png"‘, ‘"svg"‘, or ‘"jpeg"‘. Default is ‘"pdf"‘.
file_name	Optional output file name without extension. If ‘NULL‘, the plot is returned without saving.
status_df	Optional data frame containing ‘Drug‘ and ‘status_col‘.
status_col	Column in ‘status_df‘ containing status labels. Drugs with non-missing values are counted in parentheses. Default is ‘"Status"‘.
width, height	Plot dimensions. Defaults are ‘20‘ and ‘15‘.
label_size	Text size for heatmap-cell labels. Default is ‘3‘.
base_size	Base text size passed to ‘ggplot2::theme_minimal()‘. Default is ‘11‘.
heatmap_type	Heatmap layout. One of ‘"triangle"‘ or ‘"full"‘. Default is ‘"triangle"‘.
split_grouped_drugs	Logical; if ‘TRUE‘, grouped ‘Drug‘ labels are expanded using ‘drug_sep‘ after top-k selection. Default is ‘FALSE‘.
drug_sep	Separator used for grouped drug labels. Default is ‘"|"‘.
facet_cols	Optional character vector of columns used for faceting.
facet_type	Faceting mode. One of ‘"wrap"‘ or ‘"grid"‘.
facet_rows, facet_grid_cols	Optional row and column facet variables for ‘facet_type = "grid"‘.
units	Units for saved plot dimensions. One of ‘"in"‘, ‘"cm"‘, ‘"mm"‘, or ‘"px"‘. Default is ‘"in"‘.

Details

‘plot_top_k_overlap()‘ creates a heatmap of pairwise overlap among drugs with ranks less than or equal to ‘top_k‘.

When status information is available, the value in parentheses gives the number of overlapping entries with at least one status-mapped drug. Top-k sets are selected before status matching or grouped-drug expansion, so annotation does not affect ranking.

When ‘split_grouped_drugs = FALSE‘, a grouped label such as ‘"daratumumab|felzartamab|isatuximab"‘ remains one overlap unit but is matched to status information using its individual members. It therefore contributes at most one status-mapped hit. When ‘TRUE‘, grouped labels are expanded after top-k selection and individual drugs can contribute separately.

If ‘status_df‘ is omitted, a ‘Status‘ column in ‘drug_ranks_df‘ is used when available. Grouped labels in the status reference are expanded internally for matching.

Value

A ‘ggplot‘ object. The underlying overlap table is available in ‘plot$data‘.

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c(
    "daratumumab|felzartamab|isatuximab",
    "drug_b",
    "drug_c",
    "drug_d"
  ),
  Method1 = c(1, 2, 3, 4),
  Method2 = c(1, 3, 2, 4)
)

status_df <- data.frame(
  Drug = c("daratumumab", "felzartamab", "drug_c"),
  Status = c("positive", "positive", "positive")
)

p <- plot_top_k_overlap(
  drug_ranks_df = rank_df,
  status_df = status_df,
  top_k = 2,
  split_grouped_drugs = FALSE
)

p$data

## End(Not run)
```

`plot_top_k_overlap_comparison` *Compare Top-K Drug Overlap Between Two Ranking Results*

Description

Creates a heatmap showing pairwise drug overlap between ranking methods from two data frames. Drug identifiers are trimmed and compared case-insensitively. Cell labels show the total overlap and, when status information is available, the number of overlapping drugs with non-missing status as ‘total (status)‘.

Usage

```
plot_top_k_overlap_comparison(
  df_x,
```

```
  df_y,
  top_k = 100,
  drug_col = "Drug",
  methods_x = NULL,
  methods_y = NULL,
  file_type = "pdf",
  file_name = NULL,
  status_df = NULL,
  status_col = "Status",
  x_axis_title = "Methods in first result",
  y_axis_title = "Methods in second result",
  width = 20,
  height = 15,
  label_size = 3,
  base_size = 11,
  split_grouped_drugs = FALSE,
  drug_sep = "\\|",
  units = "in"
)

## S4 method for signature 'PlotObject'
plot_top_k_overlap_comparison(
  df_x,
  df_y,
  top_k,
  drug_col,
  methods_x,
  methods_y,
  status_df,
  status_col,
  x_axis_title,
  y_axis_title,
  label_size,
  base_size,
  split_grouped_drugs,
  drug_sep
)
```

Arguments

df_x, df_y	Data frames containing a shared drug identifier column and numeric ranking columns. ‘df_x‘ may also be a ‘PlotObject‘. Smaller rank values are treated as better.
top_k	Maximum rank included in each drug set.
drug_col	Name of the shared drug identifier column.
methods_x, methods_y	Optional ordered ranking columns from ‘df_x‘ and ‘df_y‘. When ‘NULL‘, all numeric columns are used.

file_type	Output format: '"pdf"', '"png"', '"svg"', or '"jpeg"'.
file_name	Optional output filename without extension. When 'NULL', the plot is returned without being saved.
status_df	Optional data frame containing 'drug_col' and 'status_col'. Drugs with non-missing status values are counted in the status overlap. When 'NULL', a '"Status"' column in either ranking input is used when available.
status_col	Name of the status column in 'status_df'.
x_axis_title, y_axis_title	Axis titles for the methods from the first and second ranking results, respectively.
width, height	Dimensions of the saved plot.
label_size	Heatmap cell-label size.
base_size	Base theme text size.
split_grouped_drugs	Logical indicating whether grouped drug labels in 'drug_col' should be split before overlap calculation.
drug_sep	Regular expression used to separate grouped drug labels.
units	Units used for 'width' and 'height'.

Value

A 'ggplot' object. Its 'data' component contains the pairwise overlap table, including total overlap, status overlap, and intersecting drug identifiers.

Examples

```
df_a <- data.frame(
  Drug = c("Drug A", "Drug B", "Drug C"),
  Method_A1 = c(1, 2, 3),
  Method_A2 = c(2, 1, 3)
)

df_b <- data.frame(
  Drug = c("drug a", "drug c", "drug d"),
  Method_B1 = c(1, 2, 3)
)

status_data <- data.frame(
  Drug = c("Drug A", "Drug B", "Drug C", "Drug D"),
  Status = c("Strong", "Weak", NA, "No effect")
)

comparison_plot <- plot_top_k_overlap_comparison(
  df_x = df_a,
  df_y = df_b,
  top_k = 100,
  status_df = status_data
)
```

```
comparison_plot
comparison_plot$data
```

`RankAggregation-class` *RankAggregation Class*

Description

A class to store results of rank aggregation analysis.

`resolve_network_inputs` *Resolve Network Inputs*

Description

Internal helper to resolve default network datasets from Synapse/cache when network inputs are omitted and to standardize/validate network schemas.

Usage

```
resolve_network_inputs(
  ppi_network = NULL,
  drug_target_network = NULL,
  group_drug_targets = TRUE,
  force = FALSE,
  auth_token = NULL
)
```

Arguments

`ppi_network`	A data frame with ‘gene1‘, ‘gene2‘ columns, or ‘NULL‘.
`drug_target_network`	A data frame with ‘ID‘, ‘Drug‘, ‘Target‘, ‘Group‘ columns, or ‘NULL‘.
`group_drug_targets`	Logical; if ‘TRUE‘, drugs sharing identical target sets are collapsed into grouped drug nodes via ‘Drug_target_reformat()‘.
`force`	Logical; if defaults are loaded from Synapse, force cache refresh.
`auth_token`	Optional Synapse token used when default network inputs need to be downloaded and no valid cache is available. If ‘NULL‘, ‘SYNAPSE_AUTH_TOKEN‘ is used.

Value

A named list containing ‘ppi_network‘ and ‘drug_target_network‘.

RRA *RRA Rank Aggregation Method*

Description

Aggregates multiple ranked drug lists using the Robust Rank Aggregation (RRA) algorithm.

Usage

```
RRA(
  object = NULL,
  input_data,
  full = TRUE,
  exact = FALSE,
  ties_method = "max",
  reverse = FALSE
)
## S4 method for signature 'RankAggregation'
RRA(object)
```

Arguments

object	Optional ‘RankAggregation‘ object. For pipe-friendly use, this can also be a data frame used as ‘input_data‘.
input_data	Data frame containing items and ranking scores. The first column should contain item identifiers, and at least two additional columns must be numeric.
full	Logical; whether to return the full RRA output. Default is ‘TRUE‘.
exact	Logical; whether to calculate exact p-values from rho scores. Default is ‘FALSE‘.
ties_method	Method used to resolve ties during rank conversion. One of ‘"max"‘, ‘"min"‘, ‘"average"‘, ‘"first"‘, ‘"last"‘, ‘"random"‘, or ‘"dense"‘. Default is ‘"max"‘.
reverse	Logical; if ‘FALSE‘, smaller input values are treated as better, which is appropriate when numeric columns already represent ranks. If ‘TRUE‘, larger input values are treated as better. Default is ‘FALSE‘.

Details

‘RRA()‘ combines rankings from at least two numeric columns in ‘input_data‘. The first column is treated as the item identifier, typically ‘Drug‘, and the remaining numeric columns are converted to ranks before aggregation.

Robust Rank Aggregation identifies items that appear consistently near the top of multiple ranked lists more often than expected by chance. The algorithm compares the observed ranks against a null model in which ranks are randomly ordered. For each item, it calculates a rho score that reflects how unexpectedly high the item appears across the input rankings.

In DrugSigNet, ranks are normalized before applying the RRA scoring procedure. The resulting rho score is reported as ‘RRA_pval‘, and items with smaller ‘RRA_pval‘ values receive better final ‘RRA_rank‘ values.

Missing values are preserved during rank conversion and ignored by the RRA scoring procedure. The final output is sorted by increasing RRA p-value.

For pipe-friendly use, a data frame can be supplied as 'object'; in that case, it is used as 'input_data'.

Value

A 'RankAggregation' object containing RRA results in 'object@result', including 'Drug', 'RRA_pval', and 'RRA_rank'.

References

Kolde R, Laur S, Adler P, Vilo J. Robust rank aggregation for gene list integration and meta-analysis. *Bioinformatics*. 2012;28(4):573-580. doi:10.1093/bioinformatics/btr709

PMID: 22247279; PMCID: PMC3278763.

Examples

```
## Not run:
rank_df <- data.frame(
  Drug = c("drug_a", "drug_b", "drug_c"),
  Method1 = c(1, 2, 3),
  Method2 = c(2, 1, 3)
)

res <- RRA(
  input_data = rank_df,
  ties_method = "max",
  full = TRUE,
  exact = FALSE,
  reverse = FALSE
)

# Pipe-friendly use
res_pipe <- RRA(rank_df, reverse = FALSE)

## End(Not run)
```

---

setup_python_dependencies

Install Python dependencies used by DrugSigNet

Description

Installs required Python modules for DrugSigNet using 'reticulate'.

By default, this installs core Python modules plus 'kaleido', which Plotly uses for static sunburst/image export. 'graph_tool' is not installed automatically because it is not available from PyPI and usually needs conda or a system package manager.

Usage

```
setup_python_dependencies(
  envname = "r-drugsignet",
  method = c("auto", "virtualenv", "conda"),
  include_graph_tool = NULL,
  force = FALSE,
  install_synapser = FALSE,
  write_renviron = FALSE,
  quiet = FALSE
)
```

Arguments

envname	Name of the Python environment to install into.
method	Installation backend passed to [reticulate::py_install()]. One of ‘"auto"‘, ‘"virtualenv"‘, or ‘"conda"‘.
include_graph_tool	Logical; whether to attempt installing ‘graph_tool‘. If ‘NULL‘ (default), DrugSigNet treats it as ‘FALSE‘. Install ‘graph_tool‘ separately with conda or a system package manager when needed. If ‘TRUE‘ and ‘method = "conda"‘, DrugSigNet installs the conda package ‘graph-tool‘ from the ‘conda-forge‘ channel.
force	Logical; if ‘TRUE‘, reinstall requested modules even if already available.
install_synapser	Logical; if ‘TRUE‘, install optional Synapse RAN support after configuring Python. This mirrors ‘DRUGSIGNET_INSTALL_SYNAPSER=true‘ in ‘tools/install_local_drugsignet.R
write_renviron	Logical; if ‘TRUE‘, persist the selected Python by writing ‘RETICULATE_PYTHON‘ to ‘~/.Renviron‘.
quiet	Logical; if ‘FALSE‘, prints progress messages.

Value

Invisibly returns a list with requested packages, missing packages prior to install, graph-tool install status, and installation method.

---

setup_synapser *Install optional Synapse support*

Description

Installs and verifies the `synapser` R package used to download DrugSigNet networks, reference databases, and annotation resources. The package is installed from the Synapse R repository after its compatible `rjson` release is installed. A GitHub fallback is available when that repository is temporarily unavailable.

Usage

```
setup_synapser(quiet = FALSE)
```

Arguments

quiet	Logical; if FALSE, show package installation progress.

Details

DrugSigNet normally calls this helper automatically when a Synapse-backed function is first used. Call it directly to install and validate Synapse support in advance.

Value

Invisibly returns TRUE after synapser has been installed and its namespace can be loaded.

Examples

```
## Not run:
setup_synapser()

## End(Not run)
```

---

targetList *Example Drug-Target Genes*

Description

An example set of target genes for target-set enrichment analysis.

Format

A character vector of gene identifiers.

TrustRank *TrustRank for Network-Based Drug Searching*

Description

Prioritizes candidate drugs or target nodes using TrustRank propagation on an integrated protein-protein interaction and drug-target network.

Usage

```
TrustRank(
  object = NULL,
  ppi_network = NULL,
  drug_target_network = NULL,
  disease_genes,
  hub_penalty = 0.01,
  damping_factor = 0.95,
  max_deg = NULL,
  result_size = NULL,
  target = "drug",
  include_indirect_drugs = TRUE,
  include_non_approved_drugs = TRUE,
  filter_paths = TRUE,
  force = FALSE,
  auth_token = NULL
)
## S4 method for signature 'NetworkBased'
TrustRank(object)
```

Arguments

object	Optional ‘NetworkBased‘ object. If ‘NULL‘, a new object is created from the supplied network inputs and method parameters.
ppi_network	Protein-protein interaction network. Expected to contain columns ‘gene1‘ and ‘gene2‘. If ‘NULL‘, the default ‘"gene_gene"‘ network is loaded.
drug_target_network	Drug-target interaction network. Expected to contain columns ‘ID‘, ‘Drug‘, ‘Target‘, and ‘Group‘. If ‘NULL‘, the default ‘"drug_target"‘ network is loaded.
disease_genes	Data frame containing disease seed genes in a column named ‘gene‘.
hub_penalty	Numeric hub penalty used to reduce the influence of highly connected nodes. Must be between ‘0‘ and ‘1‘. Default is ‘0.01‘.
damping_factor	Numeric damping factor used during TrustRank propagation. Must be between ‘0‘ and ‘1‘. Default is ‘0.95‘.
max_deg	Maximum allowed node degree. Nodes with degree greater than this value are excluded. If ‘NULL‘, defaults to ‘.Machine$integer.max‘.
result_size	Number of top-ranked results to return. If ‘NULL‘, it is inferred from the number of unique drugs in ‘drug_target_network‘.
target	Node type to prioritize. Default is ‘"drug"‘.
include_indirect_drugs	Logical; whether to include drugs indirectly connected to disease genes. Default is ‘TRUE‘.
include_non_approved_drugs	Logical; whether to include non-approved or investigational drugs. Default is ‘TRUE‘.

filter_paths	Logical; whether to restrict graph paths according to the backend implementation. Default is 'TRUE'.
force	Logical; force redownload of default networks from Synapse instead of using the local cache.
auth_token	Optional Synapse authentication token. If 'NULL', the 'SYNAPSE_AUTH_TOKEN' environment variable is used.

Details

TrustRank is a seed-based propagation method derived from PageRank. Disease genes are used as trusted seed nodes, and their signal is propagated through the integrated network to prioritize drugs or targets topologically close to the disease module.

The integrated network is built from a protein-protein interaction network ('ppi_network') and a drug-target interaction network ('drug_target_network'). If either network is 'NULL', the corresponding high-confidence default DrugSigNet network is loaded with 'load_drugsignet_network()' from the local cache or Synapse.

This implementation follows the network-based drug search setting used in CADDIE, where disease seed genes are analyzed on a combined interactome of protein-protein and drug-target interactions.

Value

A 'NetworkBased' object containing TrustRank results in 'object@result' and method parameters in 'object@parameters'.

References

Gyongyi Z, Garcia-Molina H, Pedersen JO. Combating web spam with TrustRank. In: *Proceedings of the Thirtieth International Conference on Very Large Data Bases*. 2004:576-587.

Hartung M, Anastasi E, Mamdouh ZM, Nogales C, Schmidt HHHW, Baumbach J, Zolotareva O, List M. Cancer driver drug interaction explorer. *Nucleic Acids Research*. 2022;50(W1):W138-W144. doi:10.1093/nar/gkac384

Examples

```
## Not run:
disease_genes <- data.frame(
  gene = c("ENSG00000130203", "ENSG00000142192", "ENSG00000171867")
)

res <- TrustRank(
  disease_genes = disease_genes,
  hub_penalty = 0.01,
  damping_factor = 0.95,
  target = "drug"
)

## End(Not run)
```

TSEA *Target Set Enrichment Analysis*

Description

Performs target set enrichment analysis against Gene Ontology or KEGG databases using Enrichr.

Usage

```
TSEA(
  object = NULL,
  targetList,
  source = "GO",
  ont = c("BP", "MF", "CC", "ALL"),
  Adjusted.P.value = 0.05,
  target_id_from = NULL
)
## S4 method for signature 'DrugAnnotation'
TSEA(object)
```

Arguments

`object`	Optional ‘DrugAnnotation‘ object. For pipe-friendly use, a character vector, factor, or data frame can also be supplied as ‘object‘ when ‘targetList‘ is omitted.
`targetList`	Character vector of gene symbols or other gene identifiers. Required unless supplied through ‘object‘.
`source`	Enrichment database source. One of ‘"GO"‘ or ‘"KEGG"‘. Default is ‘"GO"‘.
`ont`	Gene Ontology namespace used when ‘source = "GO"‘. One of ‘"BP"‘, ‘"MF"‘, ‘"CC"‘, or ‘"ALL"‘. Default is ‘"ALL"‘.
`Adjusted.P.value`	Adjusted p-value cutoff used to filter enrichment results. Default is ‘0.05‘.
`target_id_from`	Optional AnnotationDbi keytype or supported alias describing the identifiers in ‘targetList‘. If ‘NULL‘, targets are treated as HGNC symbols unless Ensembl gene IDs are detected automatically.

Details

‘TSEA()‘ tests whether a set of drug targets is enriched for biological pathways or functional terms. It is mainly used to summarize the biological functions represented by targets of top-ranked drugs.

For Gene Ontology enrichment, ‘source = "GO"‘ and ‘ont‘ selects Biological Process (‘"BP"‘), Molecular Function (‘"MF"‘), Cellular Component (‘"CC"‘), or all three ontologies (‘"ALL"‘). For KEGG enrichment, use ‘source = "KEGG"‘.

‘targetList‘ should contain gene symbols by default. If another identifier type is supplied, use ‘target_id_from‘ to specify the source identifier type. Supported aliases include ‘"ENSEMBL"‘ or ‘"ensembl_gene_id"‘ for Ensembl gene IDs, ‘"ENTREZID"‘ or ‘"entrezgene_id"‘ for Entrez IDs,

and ‘"SYMBOL"‘ or ‘"hgnc_symbol"‘ for HGNC symbols. Non-symbol identifiers are converted to HGNC symbols before enrichment. If ‘target_id_from = NULL‘, Ensembl gene IDs are detected automatically when possible; otherwise targets are treated as HGNC symbols.

A character vector or one-column data frame can be supplied directly or passed through ‘object‘ for pipe-friendly use.

Value

A ‘DrugAnnotation‘ object containing filtered enrichment results in ‘object@result‘. Gene Ontology results include an ‘ont‘ column identifying the BP, MF, or CC namespace.

Examples

```
## Not run:
target_genes <- c("TP53", "BRCA1", "EGFR")

res_go <- TSEA(
  targetList = target_genes,
  source = "GO",
  ont = "BP",
  Adjusted.P.value = 0.01
)

ensembl_targets <- c("ENSG00000141510", "ENSG00000012048")

res_ensembl <- TSEA(
  targetList = ensembl_targets,
  source = "GO",
  ont = "ALL",
  target_id_from = "ENSEMBL"
)

# Pipe-friendly use
res_pipe <- TSEA(ensembl_targets, ont = "ALL")

## End(Not run)
```

`validateInputs` *Validate Inputs*

Description

Validates the required inputs for the TrustRank analysis.

Usage

```
validateInputs(object)
```

Arguments

object	Optional 'NetworkBased' object. If 'NULL', a new object is created from the supplied network inputs and method parameters.

Value

Throws an error if inputs are invalid, otherwise invisible.

---

validate_network_inputs

Validate Network Inputs Before Running Pipelines

Description

Validates network analysis inputs ('ppi', 'dti', 'disease_genes') before expensive method execution. This helper checks required schemas, missing values, duplicated edges, self-loops, and disease-gene overlap with the PPI network.

Usage

```
validate_network_inputs(ppi, dti, disease_genes, strict = TRUE)
```

Arguments

ppi	A data frame with columns 'gene1' and 'gene2'.
dti	A data frame with columns 'Drug' and 'Target'.
disease_genes	A data frame with column 'gene'.
strict	Logical; if 'TRUE', overlap failures stop with an error. If 'FALSE', overlap failures emit a warning.

Value

Invisibly returns 'TRUE' when checks pass.

`write_pipeline_results` *Write Pipeline Results to Excel*

Description

Writes key tables from a DrugSigNet pipeline result object to a multi-sheet Excel workbook.

Usage

```
write_pipeline_results(result_obj, file_path, top_n = 100)
```

Arguments

`result_obj`	Result object returned by a DrugSigNet pipeline. Both the S4 pipeline objects returned by current pipeline functions and legacy list results are supported.
`file_path`	Output '.xlsx' file path. If a directory is supplied, the workbook is written as 'drugsignet_pipeline_results.xlsx' inside that directory.
`top_n`	Positive integer threshold used to create the top-ranked drug sheet. Default is '100'.

Details

'write_pipeline_results()' exports tables from 'drugNetworkPipeline()', 'drugSignaturePipeline()', or combined DrugSigNet pipeline outputs.

The workbook may include raw drug-searching results, harmonized drug rankings, drug annotations, top-ranked drugs with annotations, and functional enrichment results when these sections are available in 'result_obj'.

Sheet names are sanitized to satisfy Excel naming rules and are made unique automatically.

Value

Invisibly returns the output file path.

Examples

```
## Not run:
write_pipeline_results(
  result_obj = result,
  file_path = "DrugSigNet_pipeline_results.xlsx",
  top_n = 100
)

write_pipeline_results(
  result_obj = result,
  file_path = "results/",
  top_n = 250
```

```
)

## End(Not run)
```

`write_report` *Write DrugSigNet Analysis Report*

Description

Generates an HTML or PDF report summarizing one or more DrugSigNet pipeline results.

Usage

```
write_report(
  object,
  title = NULL,
  file = NULL,
  write_results = FALSE,
  device = c("html", "pdf"),
  interactive = FALSE,
  author = NULL,
  dashboard = c("tabset", "shinydashboard")
)
```

Arguments

`object`	A DrugSigNet pipeline result object or a list of pipeline result objects.
`title`	Optional report title. Defaults to ‘"DrugSigNet <current date>"‘.
`file`	Output file name with or without extension. Defaults to ‘"drugsignet_<current date>"‘.
`write_results`	Logical indicating whether pipeline results should also be exported as Excel files. Default is ‘FALSE‘.
`device`	Output format. One of ‘"html"‘ (default) or ‘"pdf"‘.
`interactive`	Logical; if ‘TRUE‘ and ‘device = "html"‘, compatible ‘ggplot‘ objects are rendered as interactive Plotly figures.
`author`	Optional author name.
`dashboard`	HTML report layout. One of ‘"tabset"‘ (default) or ‘"shinydashboard"‘.

Details

‘write_report()‘ creates a report from pipeline results returned by DrugSigNet workflows.

HTML reports include interactive sections for:

- Drug searching results,

- Rank aggregation,
- Drug annotation,
- Visualization.

Reports can optionally export pipeline results as Excel files using ‘write_pipeline_results()‘. PDF reports require a LaTeX engine such as ‘pdflatex‘, ‘xelatex‘, or ‘lualatex‘. If the ‘tinytex‘ R package is installed but no LaTeX engine is available, ‘write_report()‘ attempts to install TinyTeX automatically before rendering the PDF.

The visualization section is generated from every entry in ‘object@Visualization$plots‘; it is not limited to a fixed set of plot names. Optional DrugAnnotation and Visualization sections are omitted when those components are absent from the pipeline result.

HTML reports can be generated either as:

- a tabset-based R Markdown report, or
- a Shiny dashboard application.

Value

Invisibly returns the path to the generated report.

Examples

```
## Not run:
## Single pipeline result
write_report(result)

## HTML report with interactive plots
write_report(
  object = result,
  file = "DrugSigNet_Report",
  device = "html",
  interactive = TRUE
)

## PDF report
write_report(
  object = result,
  device = "pdf"
)

## Multiple pipeline results
write_report(
  object = list(result1, result2),
  dashboard = "tabset"
)

## End(Not run)
```

Index